



Cloud y Arquitecturas Distribuidas

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Modelo on-premises

- Recursos informáticos alojados en instalaciones de alguna organización
- Pros
 - Control total
 - Uso de CAPEX
 - Compliance
- Cons
 - Costos elevados
 - Flexibilidad y escalabilidad limitada



Implicancias

- Software de aplicación:
 - Escalar hacia arriba y hacia abajo: tolerancia a la desconexión (Cloud & cliente)
- Software de infraestructura:
 - Considerar costeo y tarificación de recursos (VM)
- Hardware:
 - Contenedores de hardware modulares y pequeños, energía proporcional al consumo (inactivo menos energía)



Retos sobre los datacenters

Reto de Adopción	Oportunidad
Disponibilidad	Múltiples proveedores & DataCenters
Data lock-in	Estandarización
Confidencialidad de datos y auditabilidad	Encriptación, VLANs, Firewalls; Almacenamiento Datos geográficos

Reto políticas y negocios	Oportunidad
Compartir reputación	Ofrecer servicios de protección a la reputación similar al email
Licencia software	Licencias pago-por-uso; ventas masivas a usuarios

Más retos

Reto crecimiento	Oportunidad
Cuellos de botella en transferencia de datos	FedEx-ing discos, Data Backup/ Archivo
Impredecibilidad de desempeño	Mejora en soporte VM, memoria flash, scheduling VMs
Almacenamiento escalable	Inventar almacenamiento escalable
Bugs en sistemas distribuidos grandes	Inventar debugger sobre VMs distribuidas
Escalar rápido	Inventar Auto-Scaler basado en ML; Snapshots

Origen

- SaaS
 - Se provee un servicio mediante software, no se entrega “el software”
- Montado sobre una infraestructura de hardware y software de sistemas
 - El Cloud
 - Cloud Público: El cloud está disponible como Utilidad
 - Cloud Privado: No es “cloud”



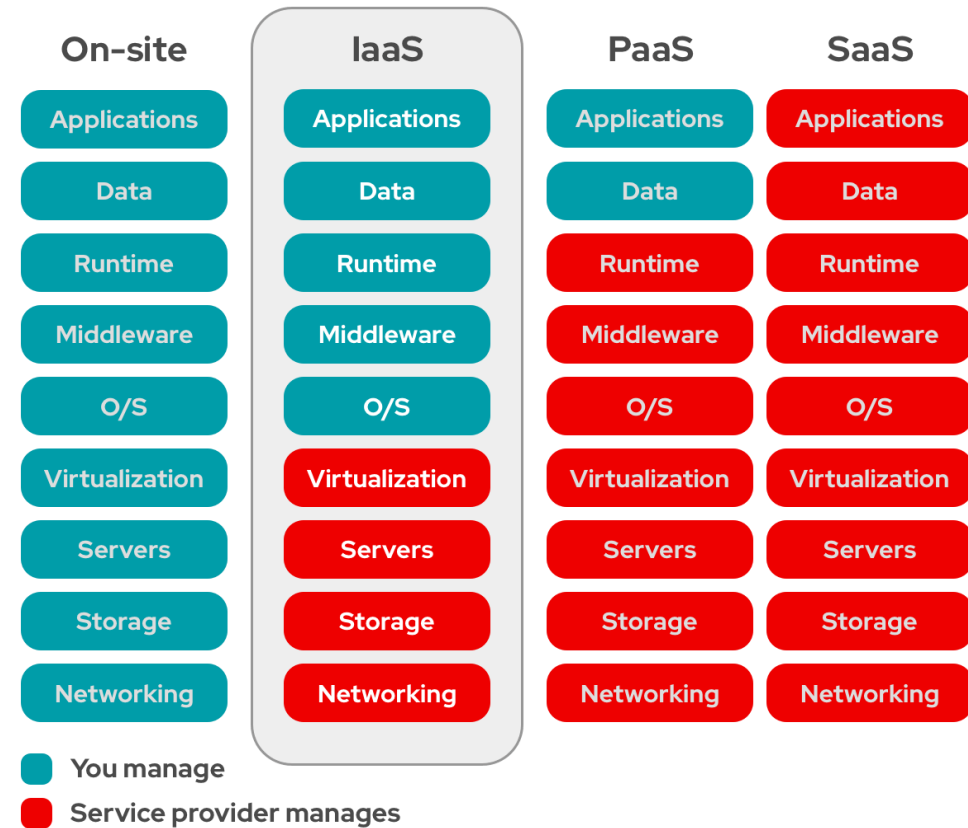


Cloud computing

- **Utilidad:** Paga por el uso
- Ilusión de recursos infinitos disponibles bajo demanda
- No hay costo fijo, se demandan recursos incrementalmente según necesidad del cliente
- Tarifa de grano fino, se paga por unidades de tiempo, memoria o almacenamiento, que también pueden dejarse de lado
- Economía de escala
- Estandarización de la pila del software

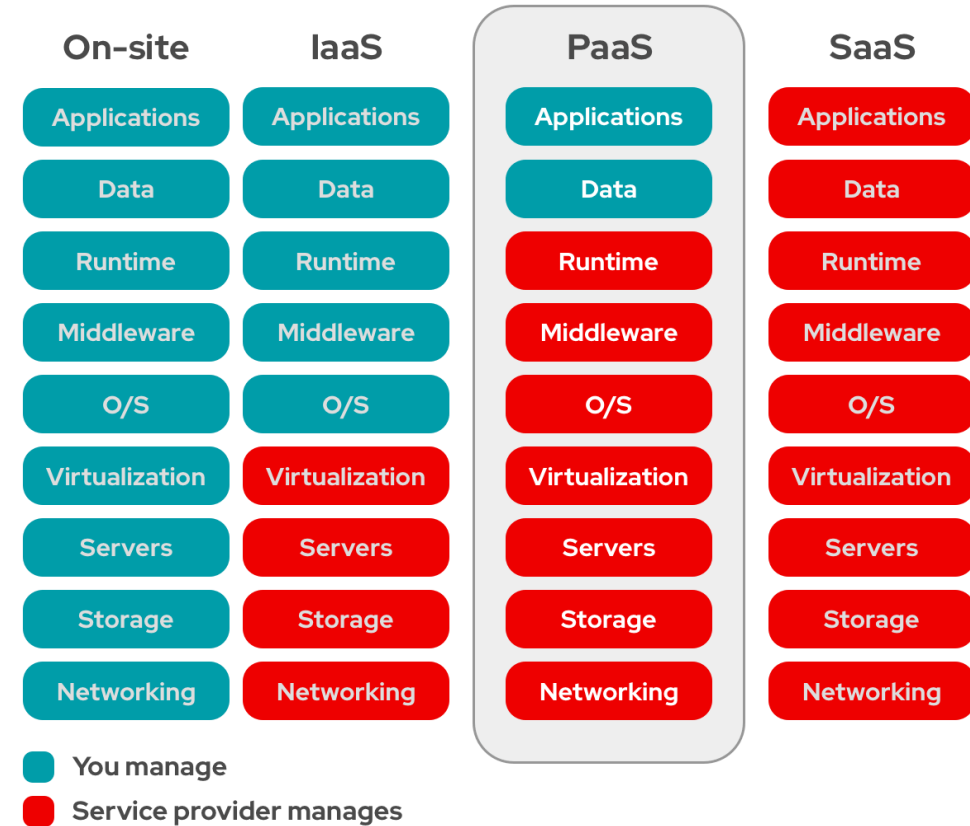
IaaS – Bajo nivel

- Virtualización de hardware
 - Pago por CPU/RAM/Disco/TPU
- Hardware configurado via API delegada
- El usuario es responsable de definir el stack de software de sistemas y aplicación: "Autoservicio"
- El Cloud provider no puede optimizar la escalabilidad ni el failover
- Requiere personal especializado
 - DevOps, Soporte, etc
- E.g.: AWS EC2, Compute engine



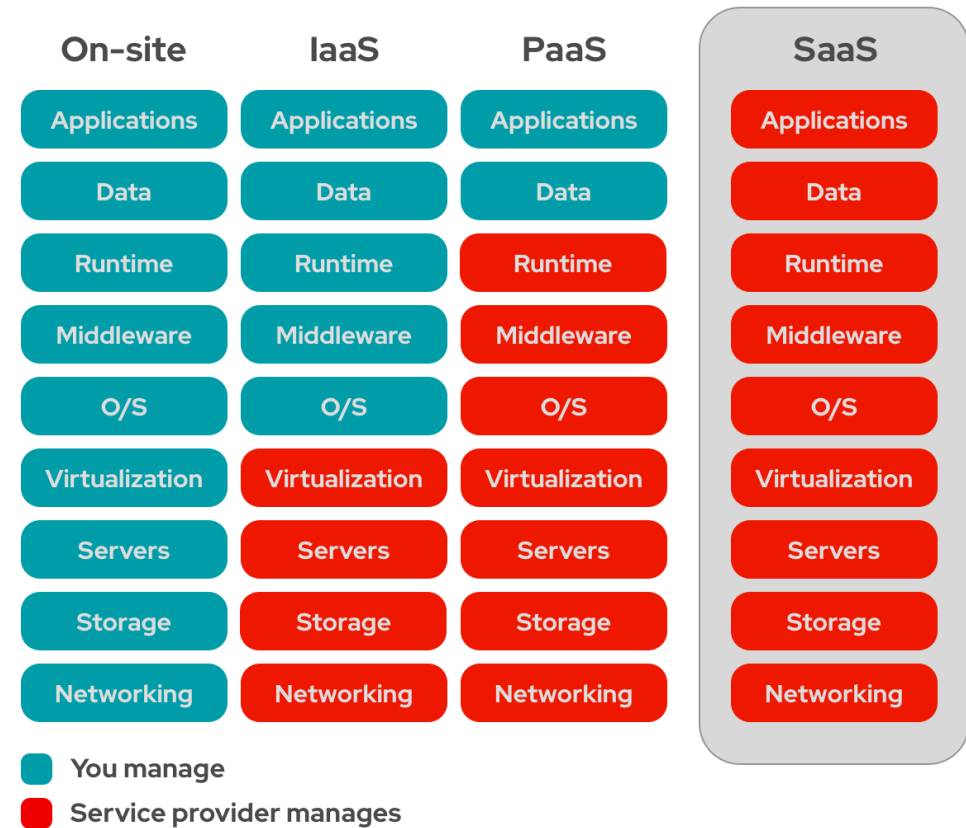
PaaS – Nivel medio

- Plataformas específicas de dominio
- Fuerzan estructura de la aplicación implementada
- Esconden complejidad al desarrollador



SaaS – Nivel alto

- Solución completa ofrecida al usuario
- Mínima modificabilidad



En AWS

Development



Lambda



API Gateway



Step Functions

Storage



S3



Glacier

Database



RDS



Dynamo



Redshift

AI



Lex



Polly



Transcribe



Comprehend



Connect

Analytics



Athena



Kinesis



Glue



QuickSight

Messaging



Pinpoint



Simple Notification
Service



Simple Email
Service

Security



Identity and Access
Management



Directory Service

Management



CloudWatch



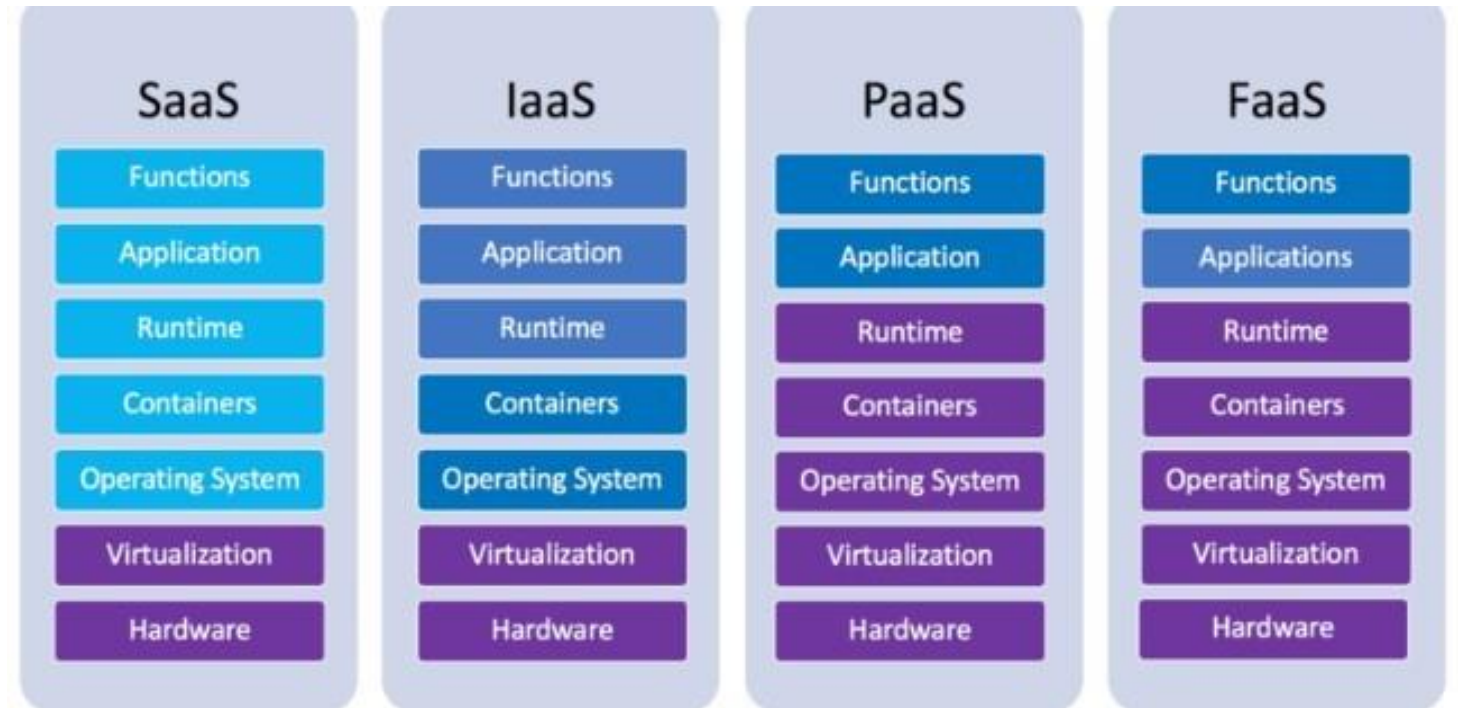
CloudFormation



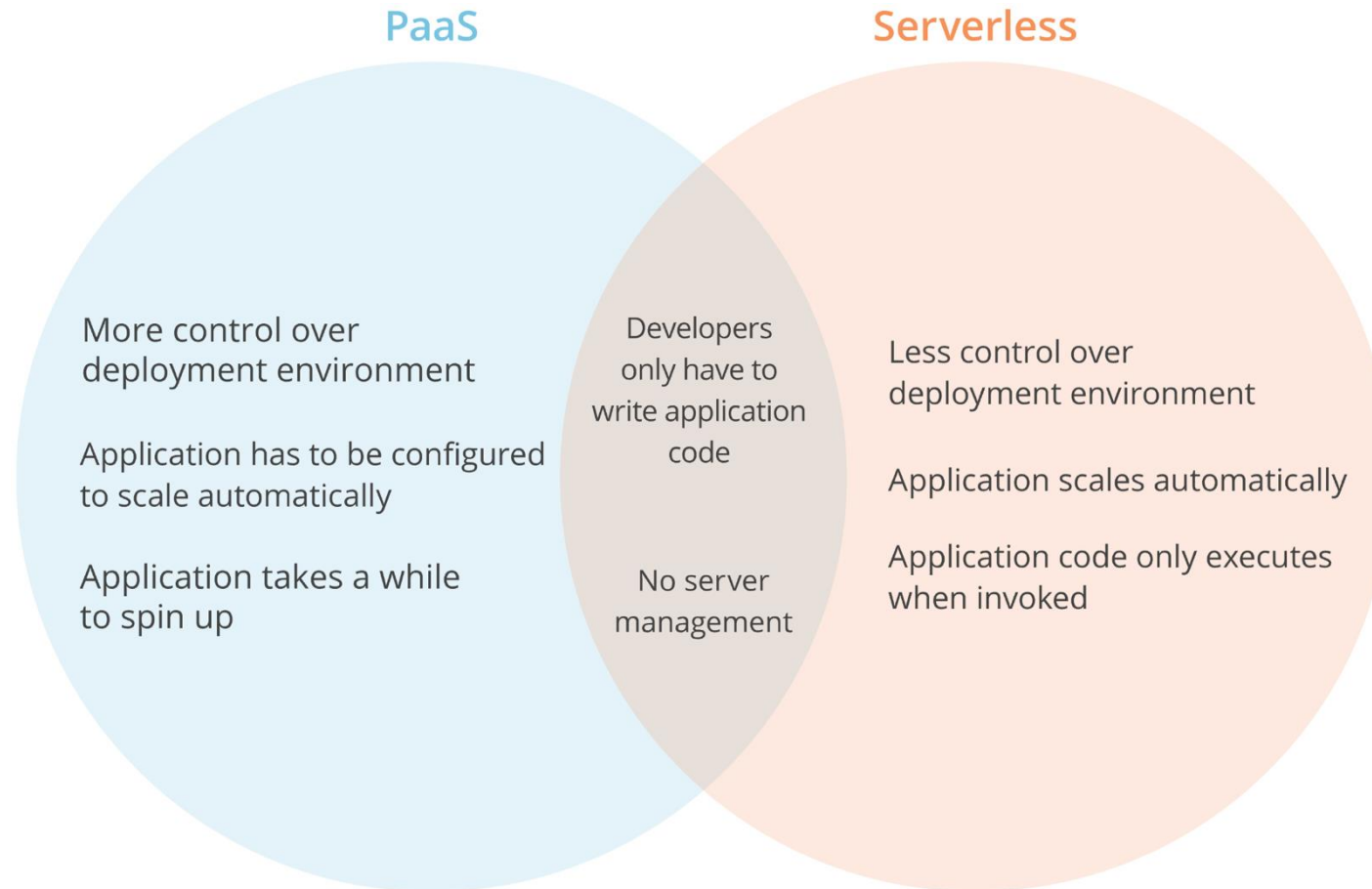
CloudTrail

FaaS & Serverless

- Serverless: Plataforma que permite desarrollar y desplegar app sin preocupación de la infraestructura
- FaaS: Ejecución de funciones, delegando infraestructura, escalamiento y aprovisionamiento en la nube



FaaS & Serverless vs PaaS



Sobre FaaS

- Pros
 - Costo: Manejo de grano muy fino,
 - Elasticidad: Escalamiento bajo demanda
- Contrás
 - Vendor lock-in
- Ejemplos
 - AWS Lambda
 - AWS SAM / Serverless.js
 - GCP cloud functions



Azure Functions

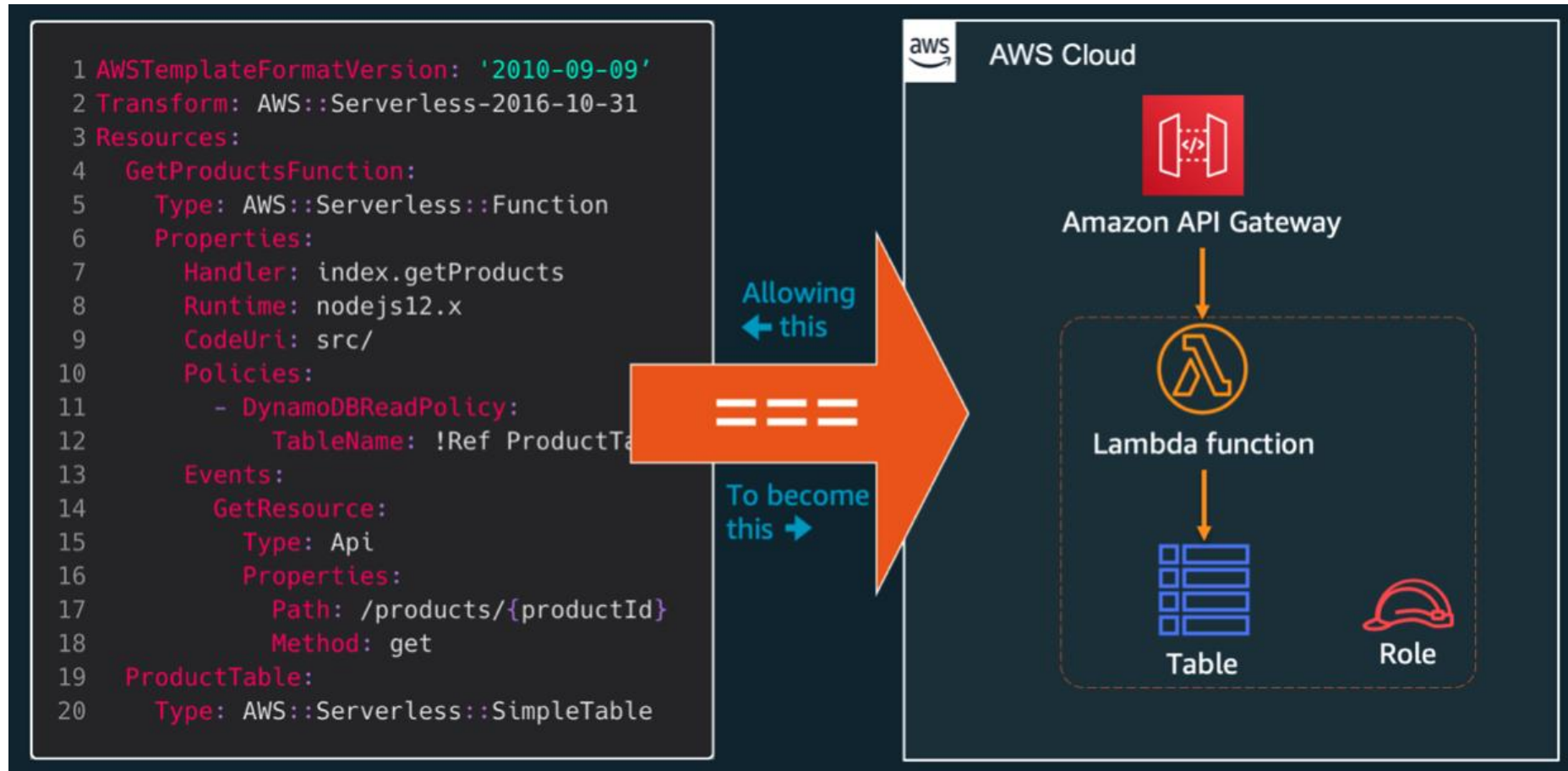


**Google
Cloud
Functions**

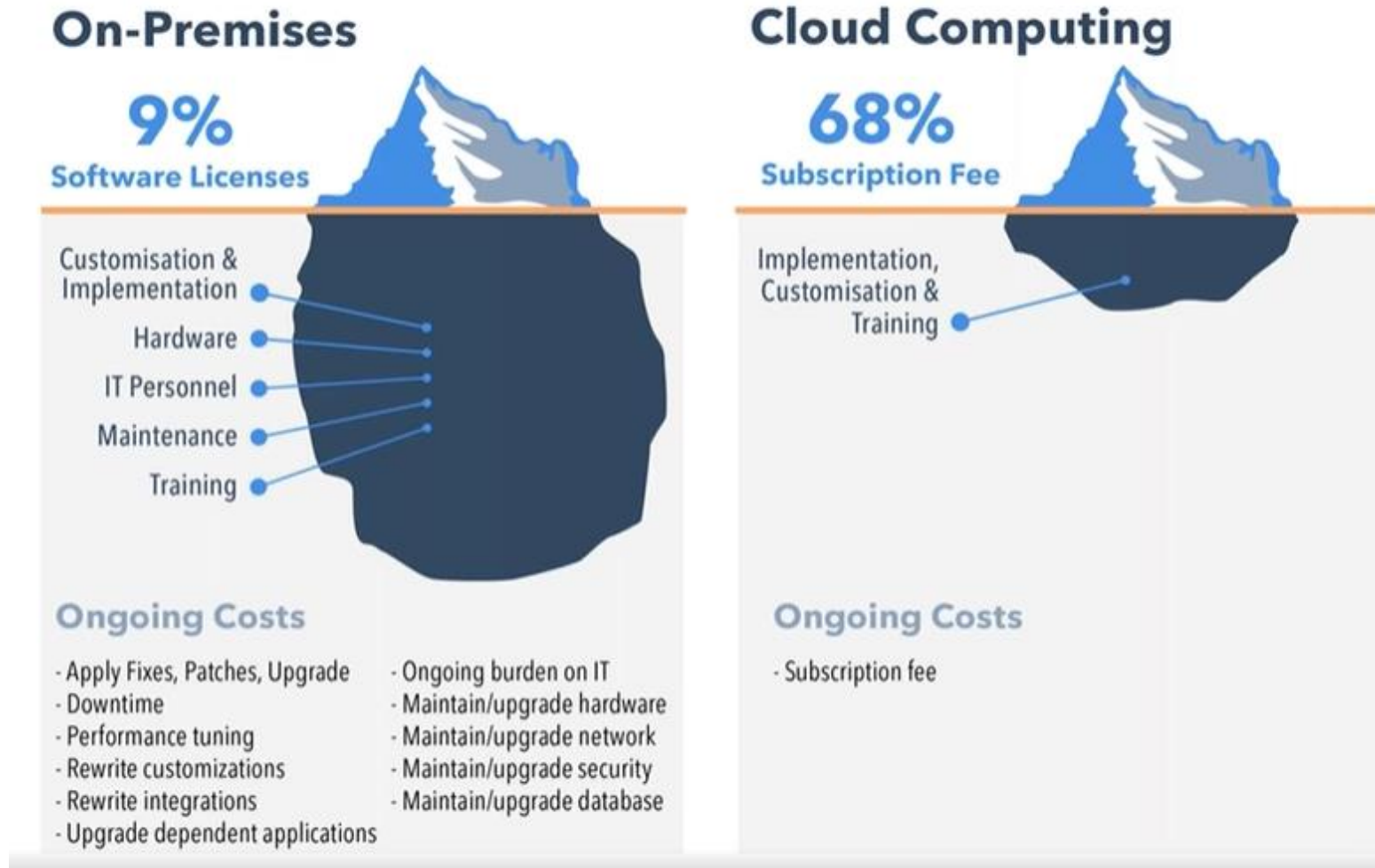


AWS Lambda

AWS SAM

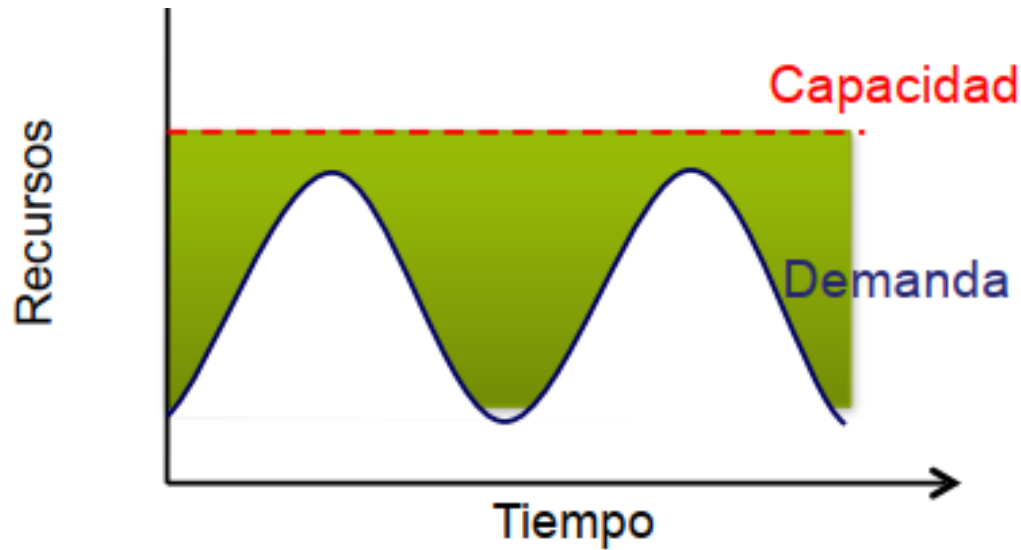


TCO: Comparativa con on-premises

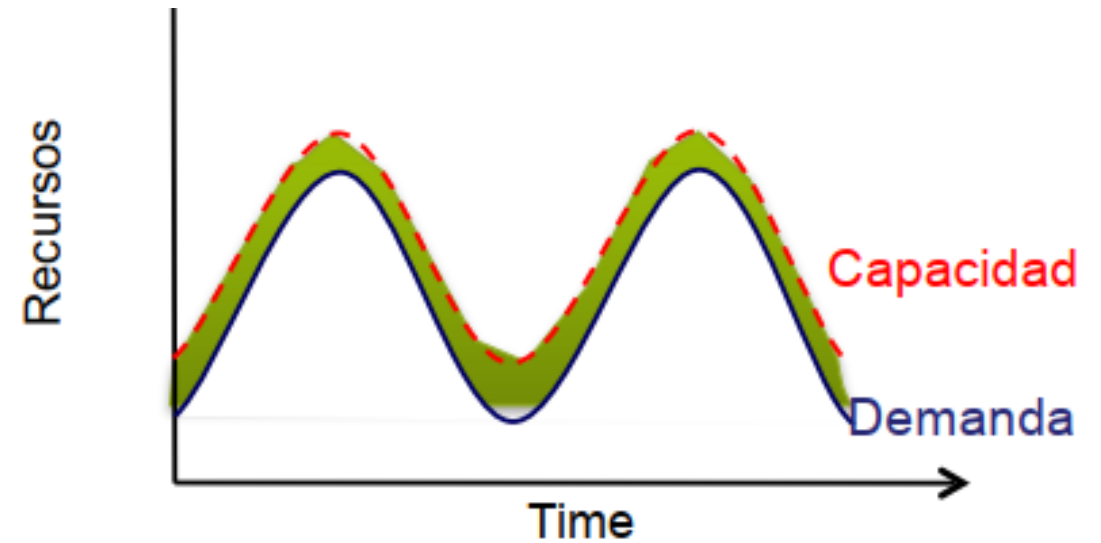


Economía en cloud: usuarios

- Pagar por el uso en lugar de aprovisionar recursos por un peak ocasional
- Menos recursos subutilizados: aprovechando elasticidad
- Transformación de CAPEX en OPEX

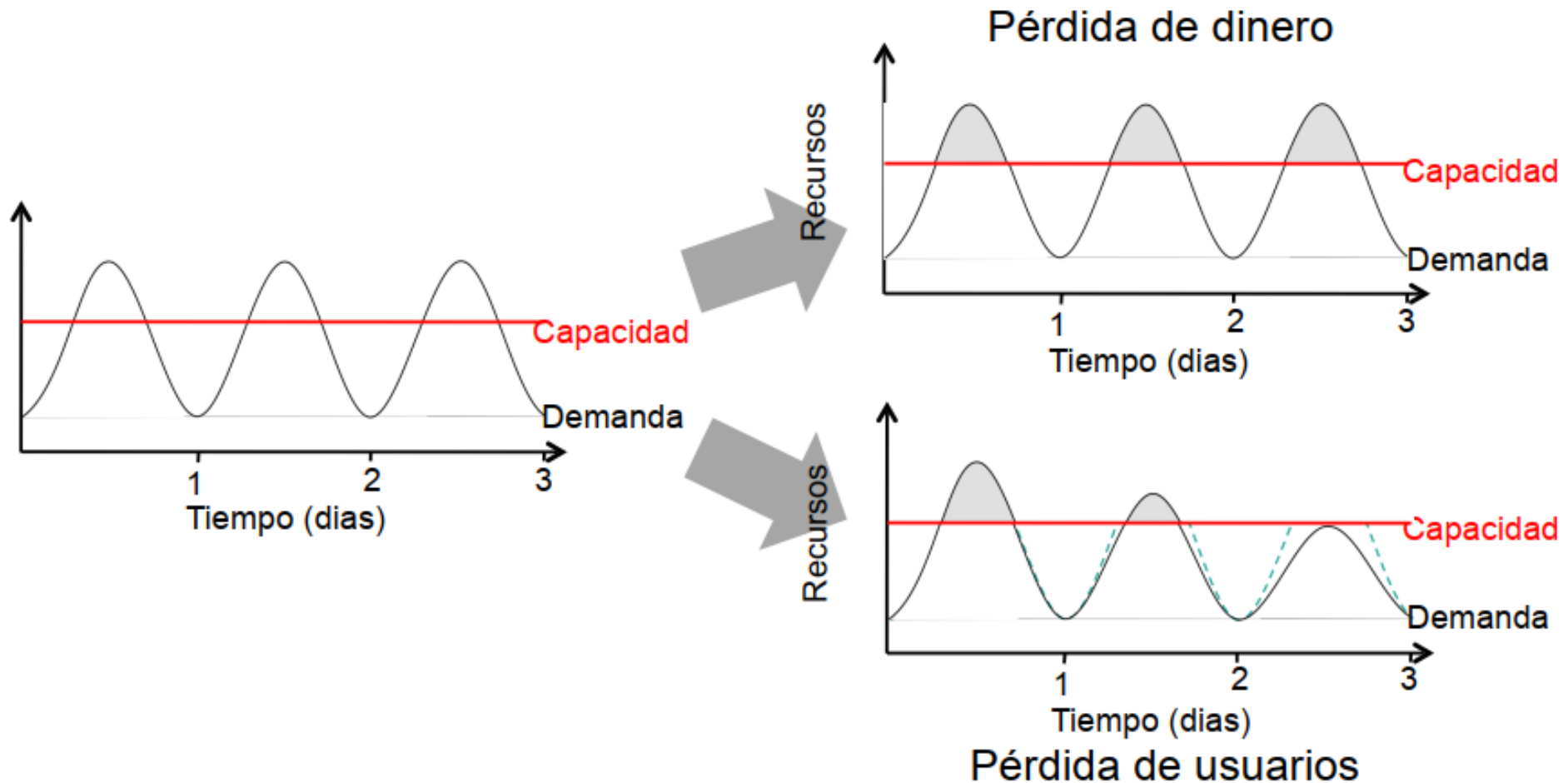


Data Center clásico



Data center en la nube

Economía en cloud: Sub-aprovisionamiento



Economía cloud: proveedores

- 5-7x economía de escala [Hamilton 2008]

Recurso	Costo Data Center Mediano	Costo Data Center Muy grande	Ratio
Red	\$95 / Mbps / mes	\$13 / Mbps / mes	7.1x
Almacenamiento	\$2.20 / GB / mes	\$0.40 / GB / mes	5.7x
Administración	≈140 servers/admin	>1000 servers/admin	7.1x

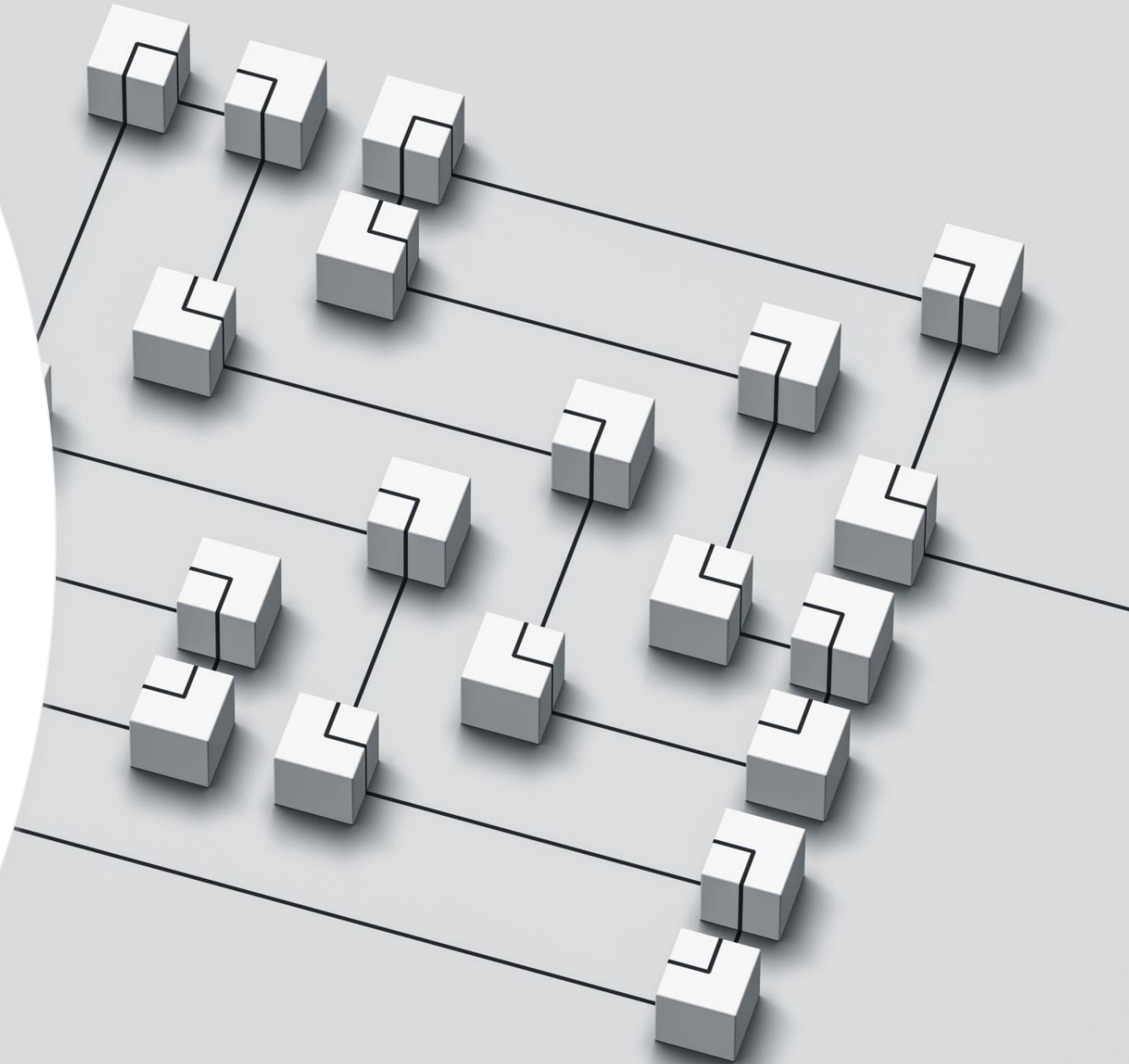


Proveedores de cloud

- Grandes actores
 - AWS - Amazon
 - GCP - Google
 - Azure – Microsoft
 - Digital ocean
 - Oracle
 - Alibaba cloud (iChina!)
- Estrategia
 - ¿Qué tengo disponible en el equipo?
 - ¿Qué usa la empresa?
 - ¿Tengo tecnología asociada?

Usos del cloud

- Data warehousing
- Procesamiento por lotes en paralelo
 - Google Map Reduce
- Analytics y BI
- Creación de SaaS
- Uso de AI, machine learning
- Levantamiento rápido de apps
 - Startups y SME (PYMEs)



Técnica de escalabilidad: Workers

- Diferentes threads en el mismo núcleo
 - Diferentes núcleos en el mismo CPU
 - Diferentes CPUs en un sistema multiprocesador
 - Diferentes máquinas en un sistema distribuido
- ¿Cómo asignamos las unidades de trabajo a los workers?

Administrar múltiples Workers

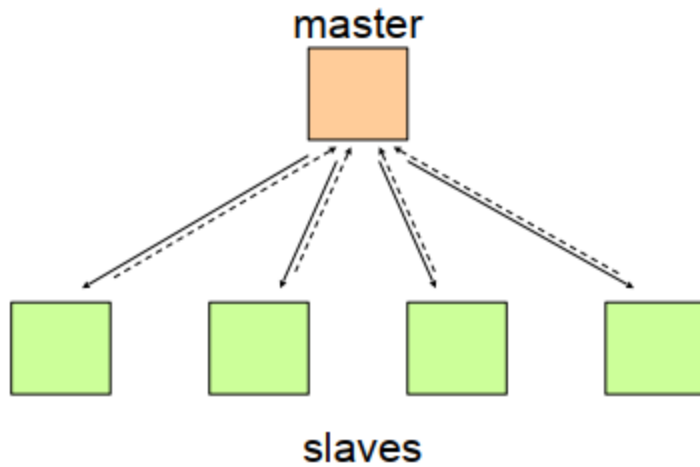
- Dificultades, generalmente:
 - No sabemos el orden en que corren
 - No sabemos dónde corren
 - No sabemos cuándo se interrumpen entre sí
 - Deadlock, livelock, race conditions, ...
 - Peor si los workers están en varias máquinas
- Necesitamos:
 - Semáforos (lock, unlock)
 - Variables condicionales (wait, notify, broadcast)
 - Barreras

Problemas de la Paralelización

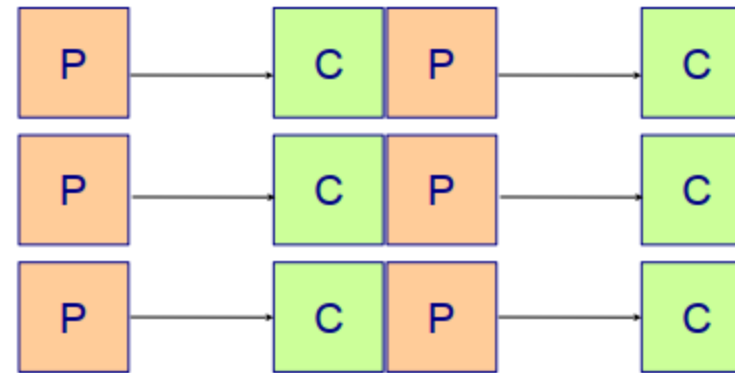
- ¿Qué pasa si tenemos más unidades de trabajo que workers?
- ¿Qué pasa si los workers necesitan compartir resultados parciales?
- ¿Cómo agregamos resultados parciales?
- ¿Cómo sabemos si todos los workers han terminado?
- ¿Qué pasa si un worker muere?
- ¡Sincronizar! ... ¿y los bugs?

Patrones de paralelismo

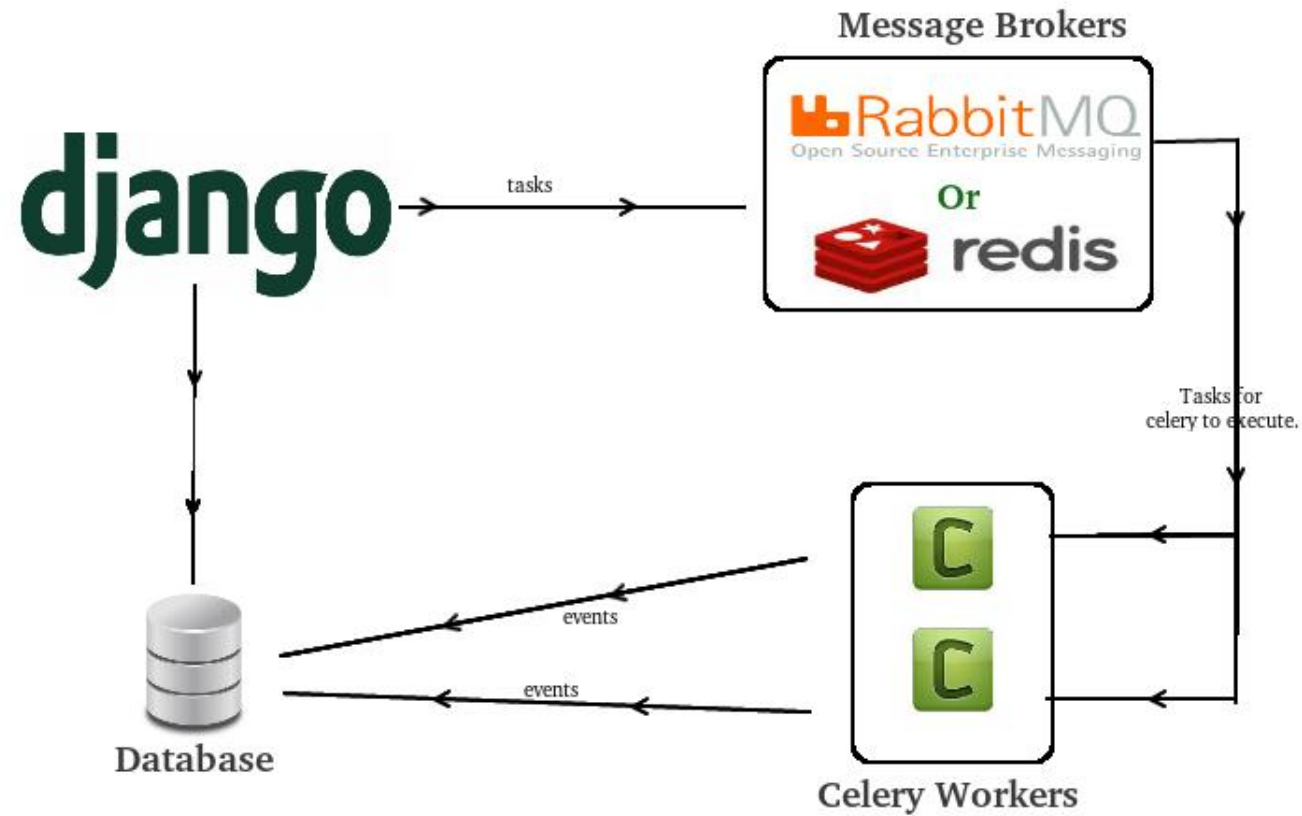
□ Master/Slave



□ Productor/Consumidor



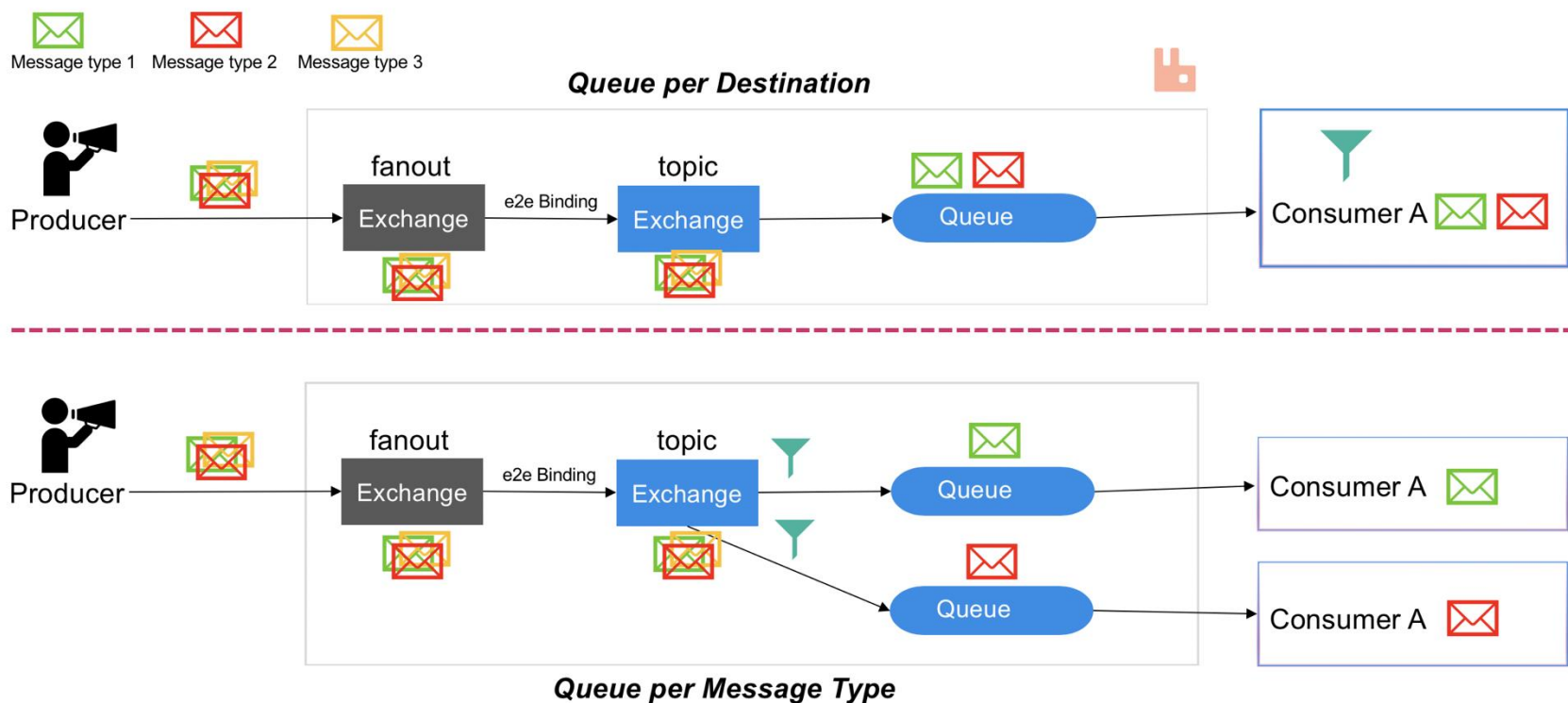
Ejemplo: Celery



Queues + paralelismo

RMQ

RabbitMQ Messaging Topology





Deployment

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Deployment



- Deployment es el proceso de montar el sistema en el hardware
 - Cambiar en tiempo de ejecución se llama “migración” o “redployment”
- Actualmente los grandes sistemas evolucionan continuamente

Proceso

- Como ya sabemos:
 - Un sistema puede estar en varios "host"
 - Dependiendo de las configuraciones y las necesidades de los componentes del sistema, estos pueden compartir:
 - Hardware
 - Network
 - Otros dispositivos
 - Datos
- Proceso:
 - Antes: mediante CD, todo el sistema se actualiza a la vez
 - Ahora: Instalación integral del sistema, con actualizaciones frecuentes

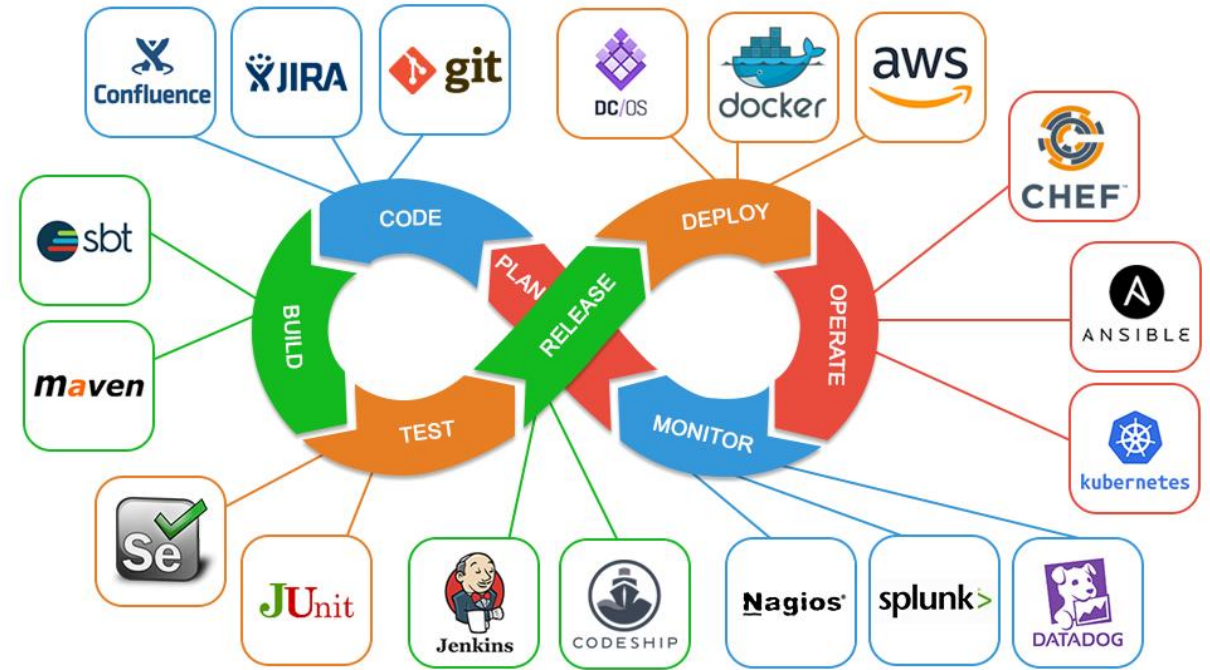
Automatización

Algunas buenas prácticas:

- Utilizar un repositorio
- Automatizar el “build”
- Tests automáticos al crear el build
- Todos hacen commits diarios
- Cada commit debe provocar un build
- Mantener el build “rápido”
- Testear un clon del ambiente de producción
- Facilitar el acceso a las versiones
- Visibilidad de los resultados de los builds
- Automatizar deployment

Herramientas deployment

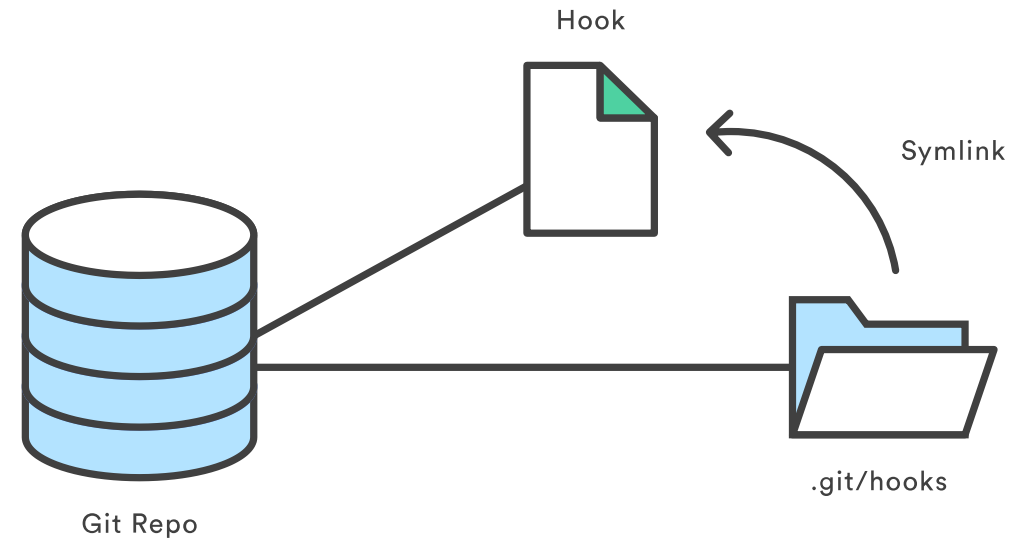
- ¿Por qué?
 - Reducción del tiempo de entrega
 - Reducción de errores
 - Repetibles (se usa una vez, se ejecuta muchas)
 - Cualquier miembro del equipo puede realizar un deployment
 - Mayor tiempo utilizado en desarrollo
 - Realizar deployment a un nuevo sistema resulta más sencillo
 - Deploys más frecuentes



Herramientas deployment: Git-Hook

- Configurar un servidor para ejecutar un script tras actualizarse el código
- Por ejemplo:
 - Correr migraciones
 - Instalar dependencias
 - Etc...
- <https://git-scm.com/book/ms/v2/Customizing-Git-Git-Hooks>

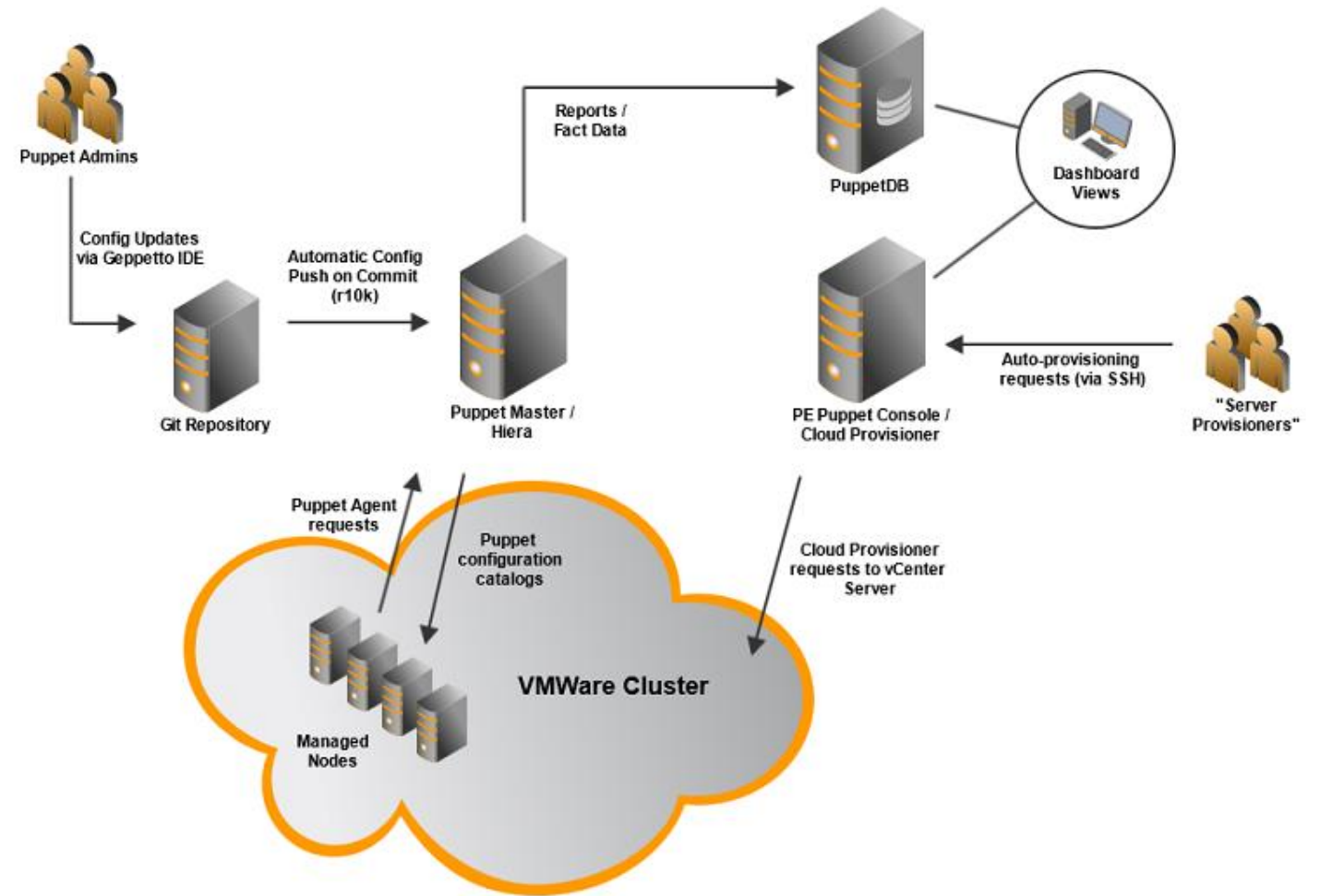
Maintaining a hook using a symlink to version-controlled script



Herramientas deployment: Puppet

Para configurar el software de nuestras máquinas automáticamente

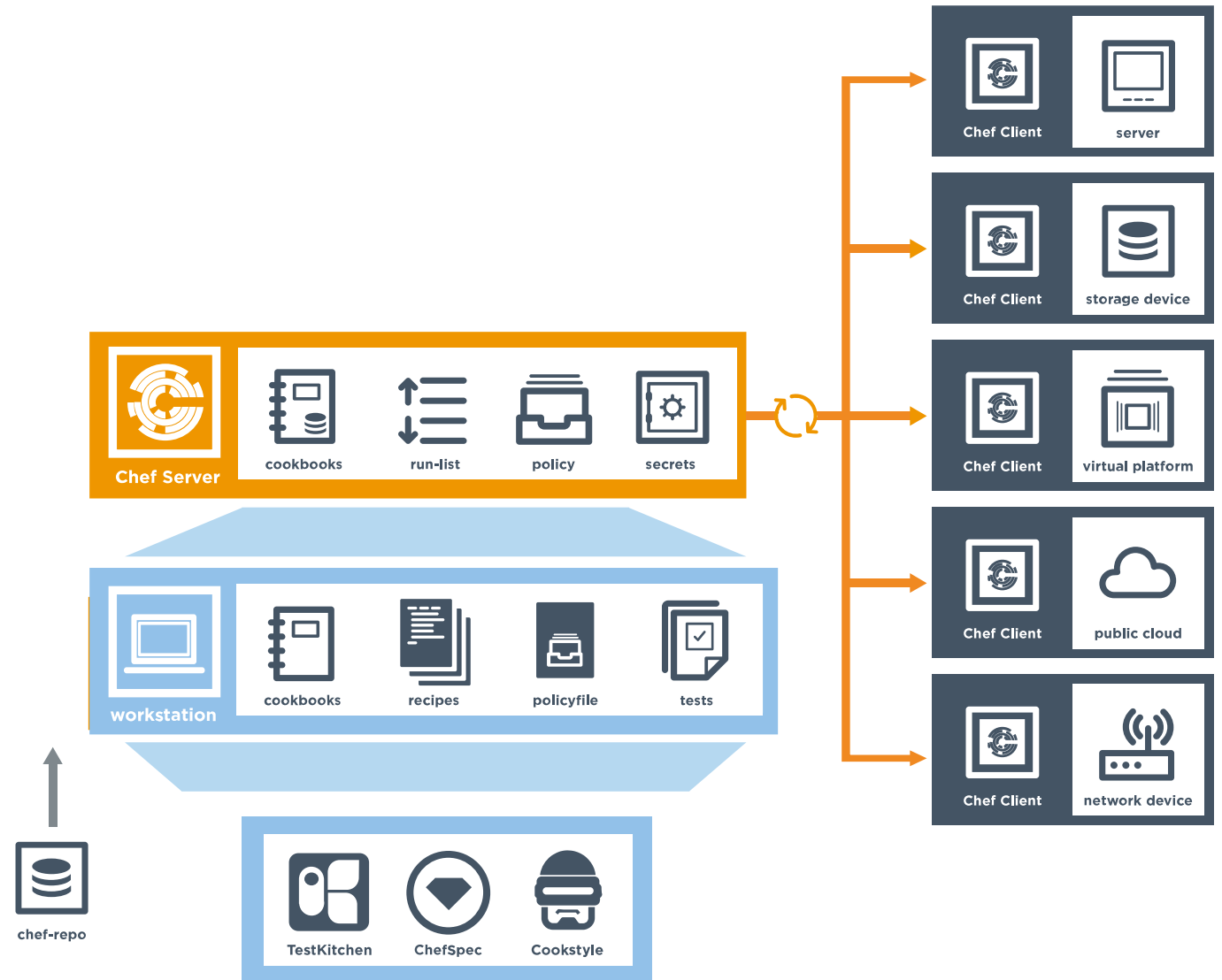
- Una de las herramientas más antiguas, maduras y estables
- Pros
 - Basado en Ruby, pero usa DSL parecido a JSON
 - Tiene una estructura cliente servidor
 - Interfaz madura y corre casi en cualquier sistema operativo
- Cons:
 - No está diseñado para ser simple
 - Gracias al uso del DSL el código crece muy rápido en proyectos complejos



Herramientas deployment: Chef

Para configurar el software de nuestras máquinas automáticamente

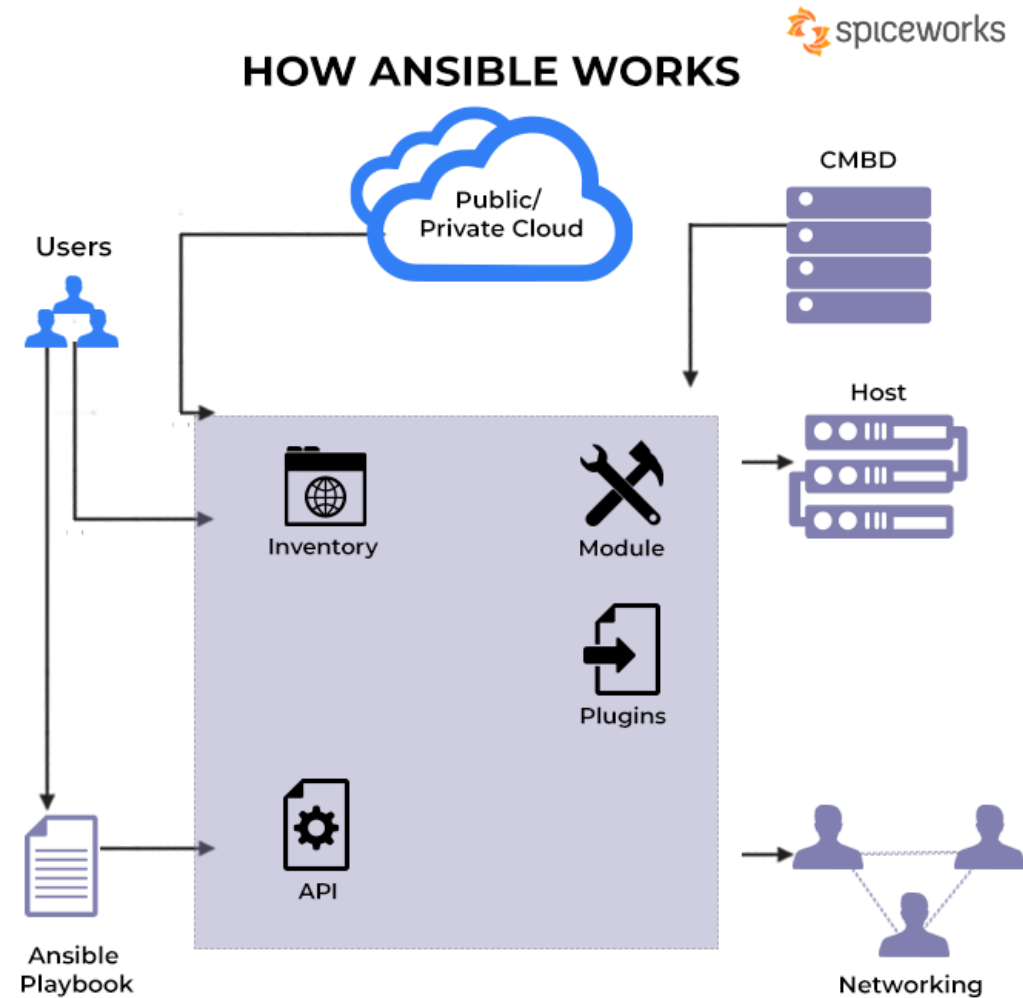
- Basado en Ruby y Erlang, Ruby DSL para declaración de recetas
- Pros
 - Tiene una estructura cliente servidor
 - Interfaz madura y corre casi en cualquier sistema operativo
 - Muy similar a Puppet
 - Muchas recetas de módulos creadas por la comunidad
- Cons:
 - Difícil de aprender si se tiene poco manejo de Ruby y de programación procedural



Herramientas deployment: Ansible

"Ansible is a radically **simple IT automation system**. It handles **configuration-management**, **application deployment**, **cloud provisioning**, **ad-hoc task-execution**, and **multinode orchestration** - including trivializing things like zero downtime Rolling updates with load balancers..."

- Desarrollada en python, pero soporta extensión de módulos en cualquier lenguaje
- <https://github.com/ansible/ansible>
- <https://github.com/ansible/ansible-examples/tree/master/mongodb>



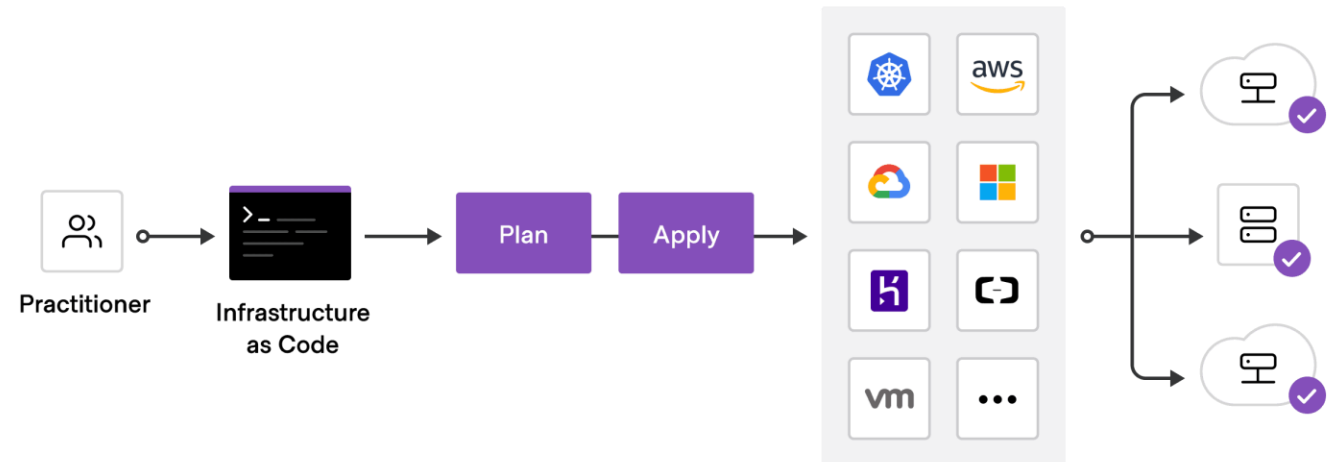
Herramientas deployment:

Algunos casos de uso

Ansible					
Puppet					
Chef					

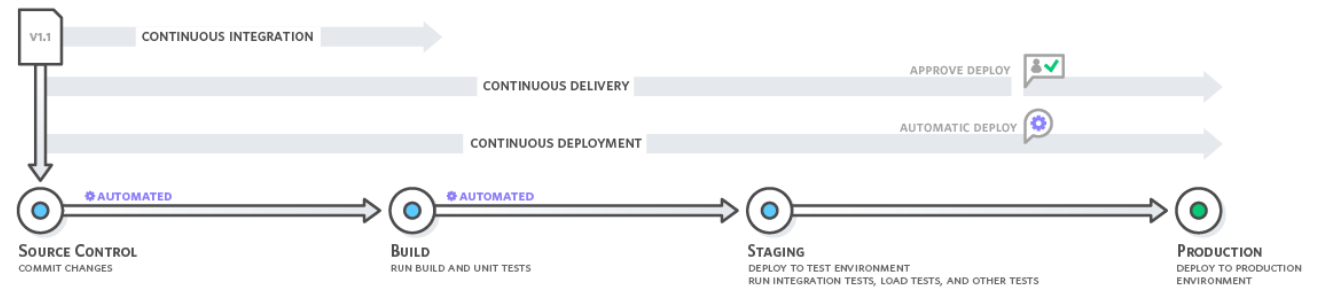
Infrastructure as Code

- Poder generar y respaldar la infraestructura de computación usando código en vez de configuraciones manuales
- Pros
 - Puede generar/duplicar entornos fácilmente
 - Menos trabajo repetitivo y manual, ¡Menos errores!
- Cons:
 - Requiere conocimientos de Arquitectura de Sistemas



Continuous Integration

- Práctica de fusionar cambios de código y ejecutar compilaciones y pruebas automatizadas de manera regular
- Pros
 - Más fácil de detectar errores en etapas tempranas
 - Testing automático
 - Evita errores de último minuto
 - Al haber poco código entre cada build, es más fácil de arreglar en caso de fallas
 - Siempre hay builds disponibles para release/testing/demo



Continuous Integration

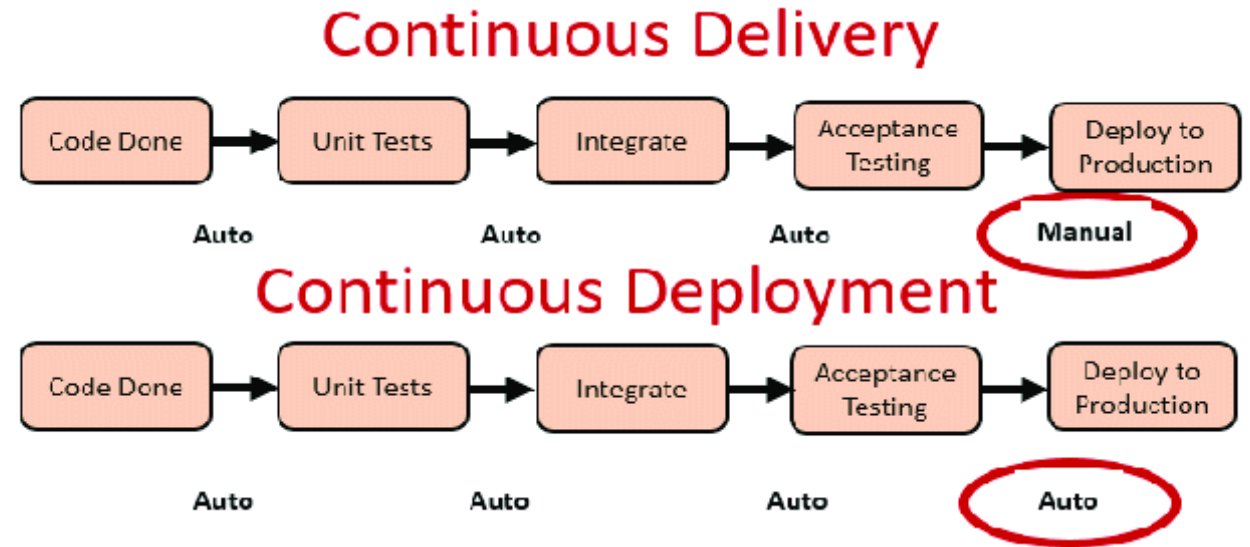
“Continuous Integration doesn’t get rid of bugs, but it does make them dramatically easier to find and remove.”

- Martin Fowler, Chief Scientist, ThoughtWorks



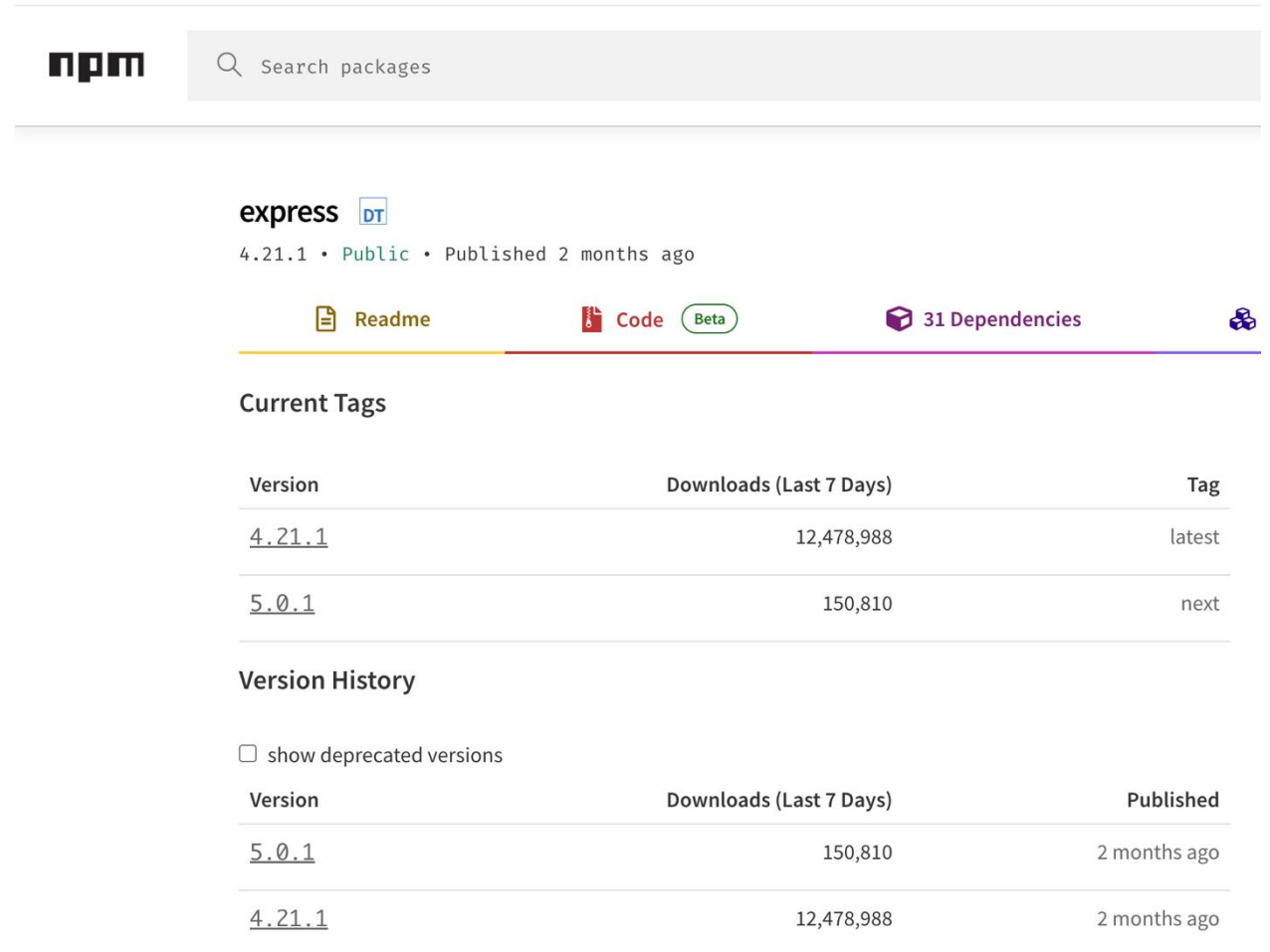
Continuous Delivery/Deployment

- Continuous Delivery:
 - Si los tests corren constantemente y hay confianza en ellos, entonces es posible hacer releases en cualquier momento
- Continuous Deployment:
 - Entregar los releases al cliente apenas el software esté listo, como en SaaS



Versionamiento

- Proceso de asignar un nombre único (versión) a cada estado del software
- No hay un estándar universal, pero...
- <https://semver.org/> (Semantic Versioning – Por Tom Preston-Werner)
 - Recomienda versiones con formato **X.Y.Z**
 - **X: MAJOR number.** Sólo debe aumentar cuando la nueva versión se hace incompatible con las otras
 - **Y: MINOR number.** Aumenta cuando se agregan funcionalidades retro-compatibles
 - **Z: PATCH number.** Aumenta cuando se corrigen bugs de versiones anteriores
 - Además, se recomienda agregar un último término, que tenga relación con el build del código



The screenshot shows the npm package page for 'express'. At the top, there's the npm logo and a search bar. Below that, the package name 'express' is displayed with a 'DT' icon. The version '4.21.1' is shown, along with 'Public' status and 'Published 2 months ago'. There are links for 'Readme', 'Code', and 'Beta', and a note about '31 Dependencies'. A section titled 'Current Tags' contains a table with two rows: '4.21.1' (latest) and '5.0.1' (next). Below this is a 'Version History' section with a checkbox for 'show deprecated versions' and a table with two rows: '5.0.1' (published 2 months ago) and '4.21.1' (published 2 months ago).

express DT

4.21.1 • Public • Published 2 months ago

[Readme](#) [Code](#) [Beta](#) [31 Dependencies](#)

Current Tags

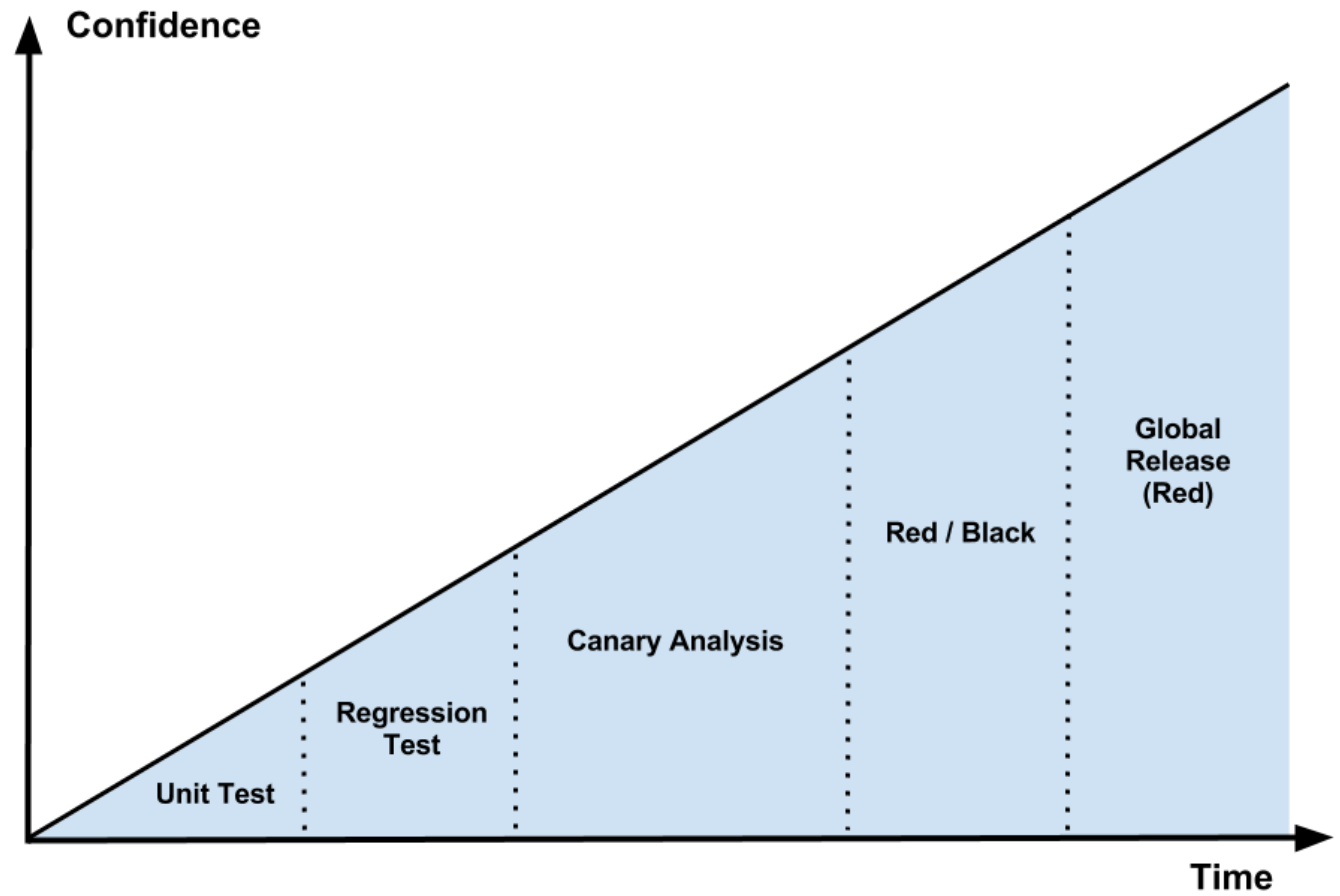
Version	Downloads (Last 7 Days)	Tag
4.21.1	12,478,988	latest
5.0.1	150,810	next

Version History

☐ show deprecated versions

Version	Downloads (Last 7 Days)	Published
5.0.1	150,810	2 months ago
4.21.1	12,478,988	2 months ago

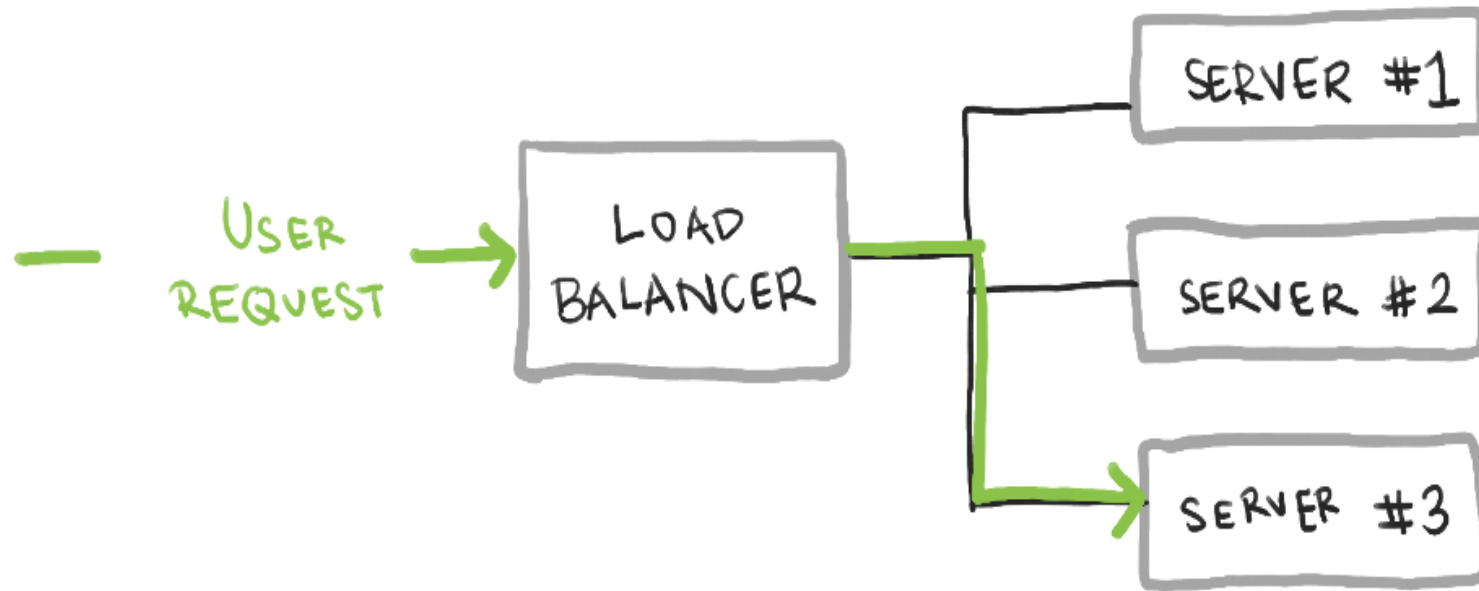
Estrategias - Netflix



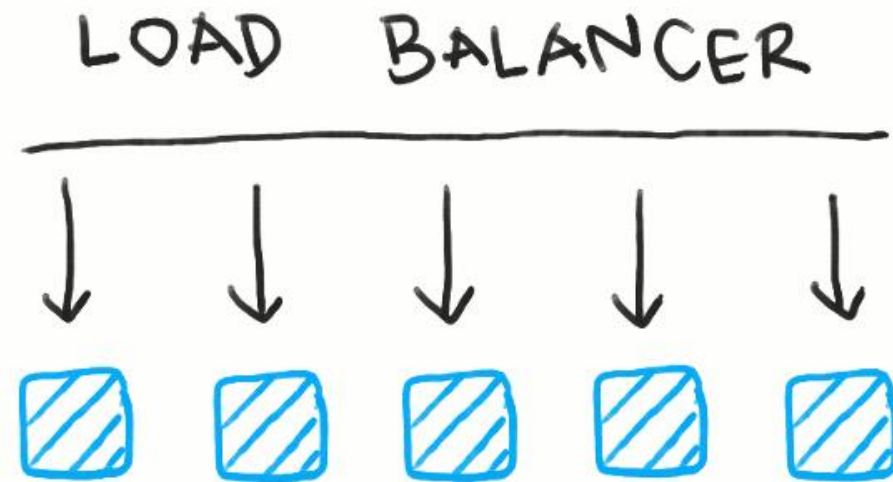
Cuando un servidor se apaga

SERVER 

Una opción

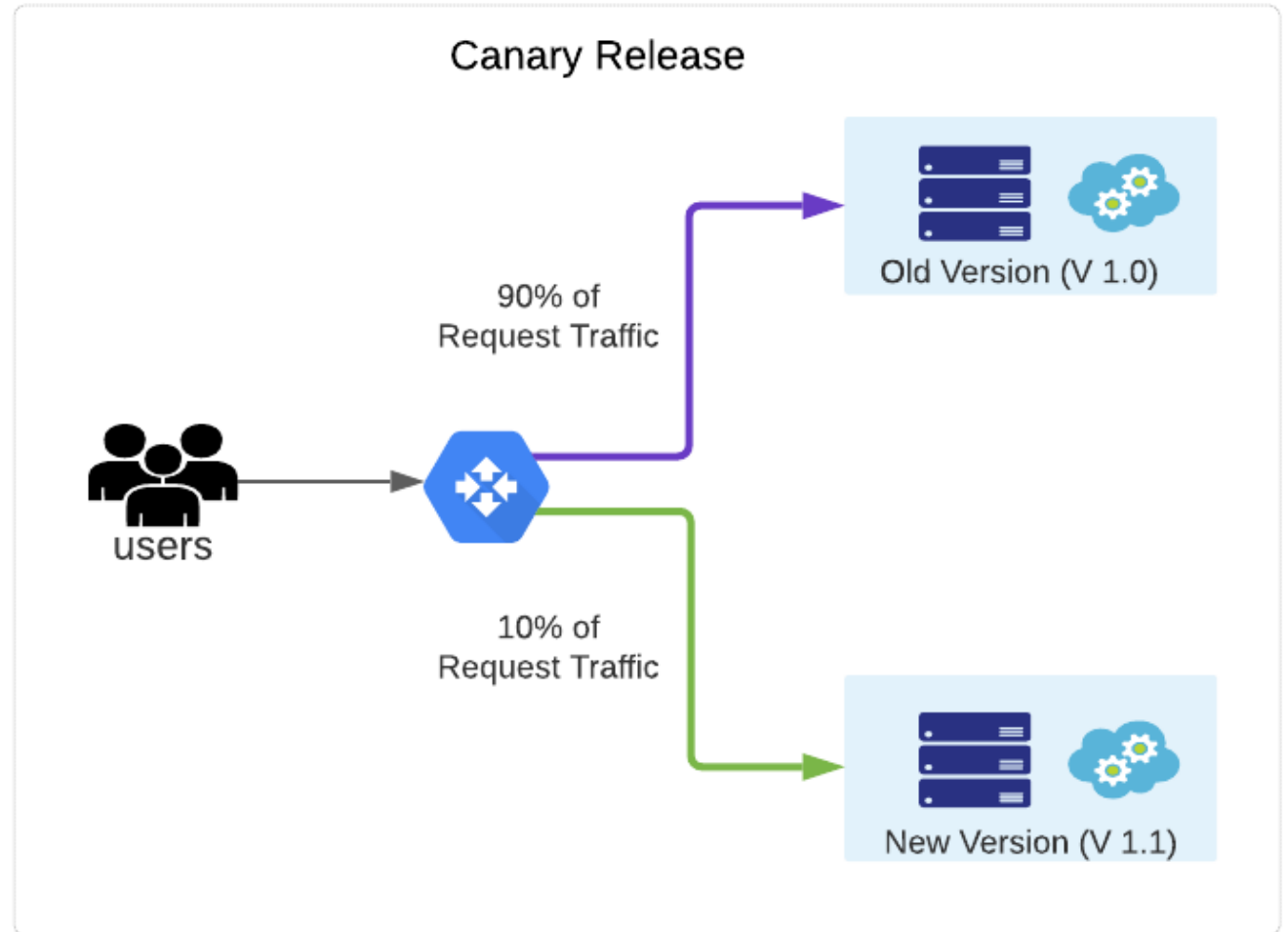


Una opción

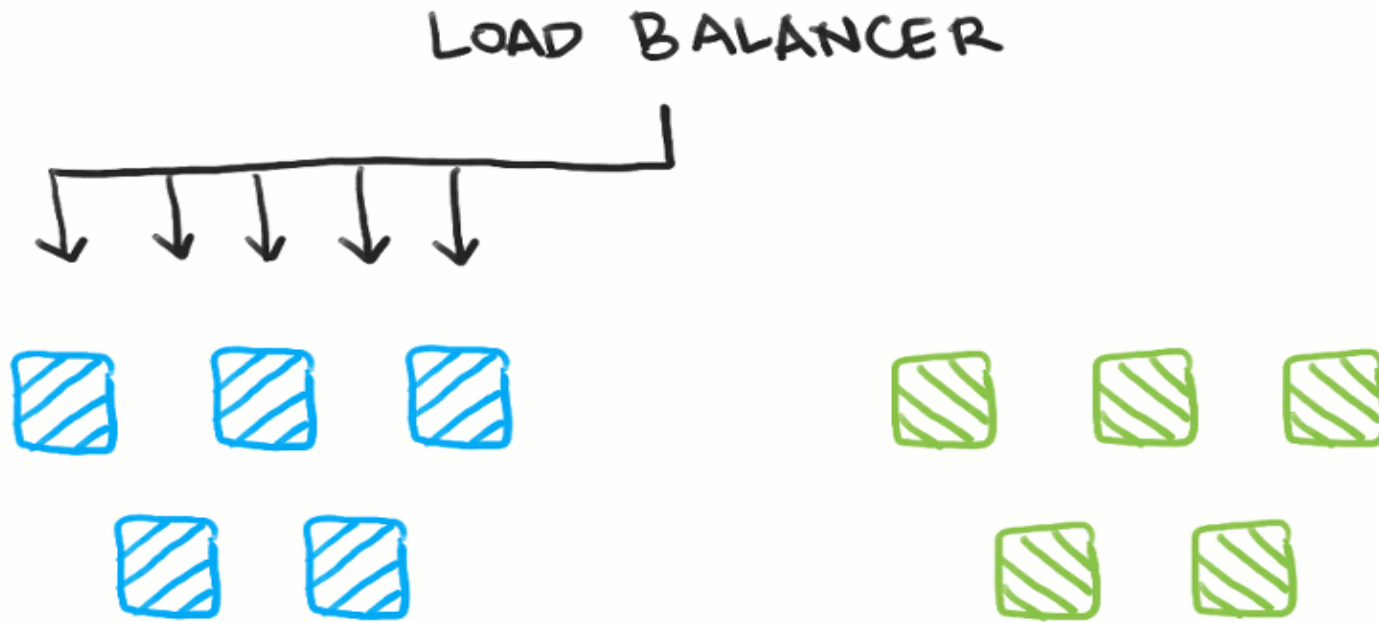


Estrategias: Canary Release

- Minimiza el impacto de una falla
- No hay mejor "tester" que los mismos usuarios
- Desafíos:
 - Manejar múltiples versiones de software
 - Integrar diariamente
 - Enfoque ágil
 - Monitoreo y control
 - A/B Testing

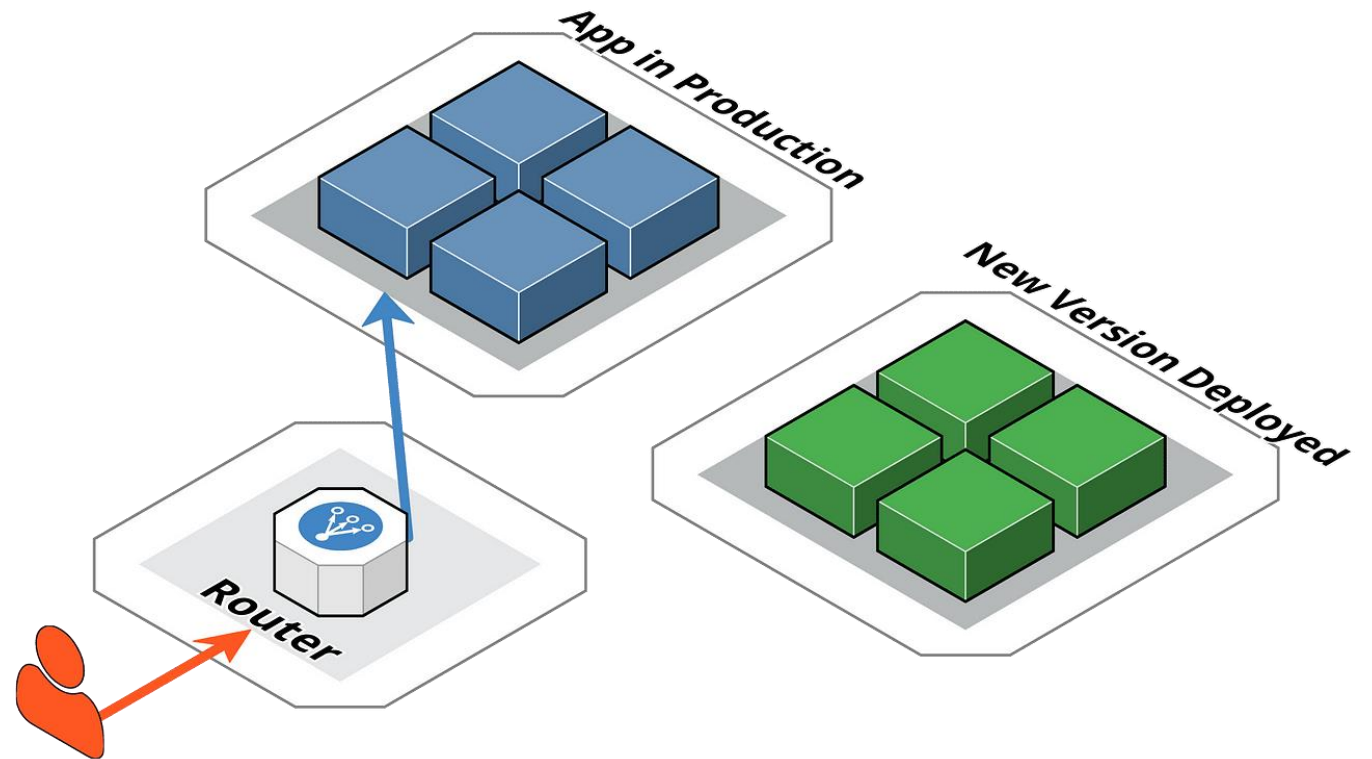


Una segunda opción



Blue/Green Deployment

- Tener ambas versiones (nueva y antigua) ejecutándose en distintos ambientes
- Stateless
 - Cookies – Archivos Caché
 - Servicios externos
- Database Migrations
 - DB compartida
 - Cambios no afecten al servidor online
- Non destructive migrations
 - Eliminación de tabla
 - Verificar que no afecte a la aplicación
 - Lanzamientos chicos y frecuentes
- Dark launches
 - Sin cambios en el UI
 - Observar comportamientos en el servidor





Sobre las Arquitecturas

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

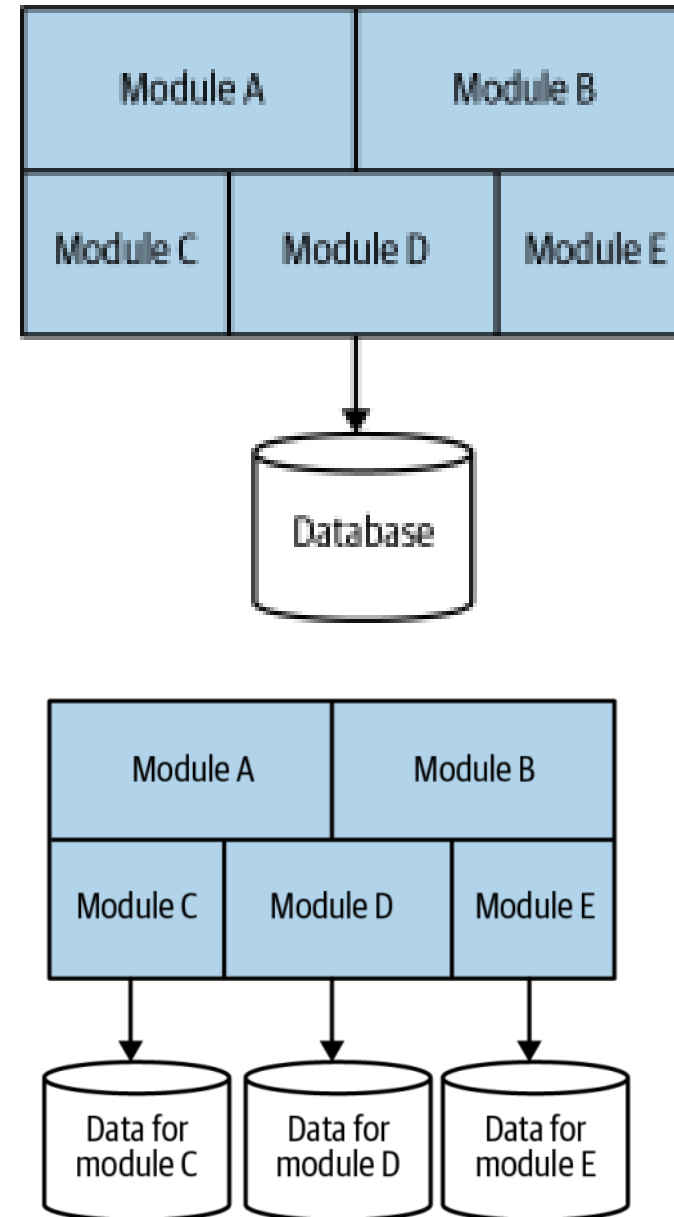
Sobre el monolito

- Sistema contenido en una sola quanta
- Un bloque de software, una base de datos
- Alta cohesión, acoplamiento estático, aunque aprecia modularidad
- Implicancias dependientes del estilo, no es inherentemente malo (debatible)



Clases de monolito

- Single process
- Modular
 - Intento de organizar funcionalidades
 - Muy común en organizaciones en maduración o en diversificación
- Distribuido
 - Productos diversificados pero inmaduros
 - Se usan instancias separadas para los módulos pero un acoplamiento previene independencia



Monolito: ¿Es la única opción para partir?

- No necesariamente
 - Muy útil para ambientes de desarrollo rápido
 - Trade-off de tener una única quanta
- Que pasa con la escalabilidad
- Que hay de dominios de negocio más complejos?
- En su proyecto ya están tomando una arquitectura más distribuida
 - ¿Cómo coordinamos todo?



Arquitecturas distribuidas

- Sistemas compuestos por más de una quanta
 - Heterogeneidad
 - Despliegue independiente
- Coordinación toma importancia
 - Orquestación
 - Coreografía



Variantes de arquitecturas distribuidas

- SoA
 - SoA + ESB
- Microservicios
- Space-based
- Workers
- Otros...



Pros

- Escalabilidad
 - Heterogeneidad + portabilidad asegurada
- Confiabilidad
 - Redundancia
 - Failover
 - Replicacion
- Flexibilidad
 - Granularidad
- Performance



Cons

- Complejidad
- Performance
 - ¿Qué pasa con operaciones cross?
- Consistencia
 - ¿Transacciones?
- Seguridad
- Costos





Cierre

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot



Que vamos aprendiendo del curso

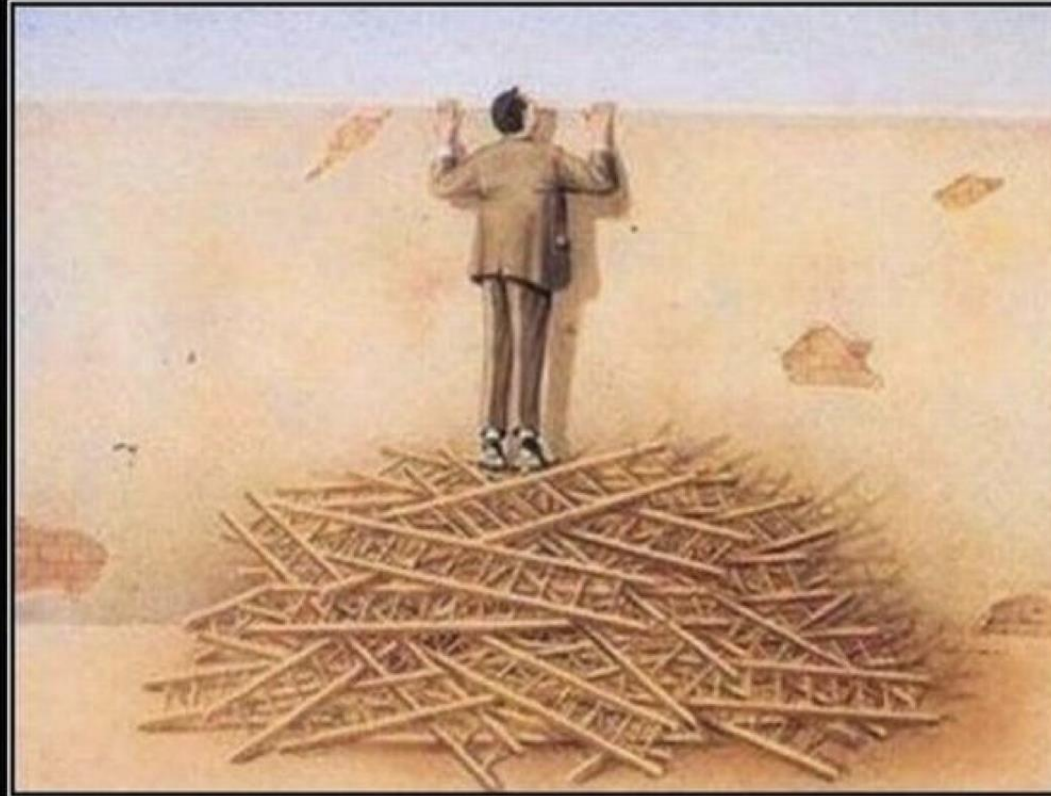
- Modelación de soluciones a problemas no tan rígidos
- Conversar situaciones hipotéticas
- Priorizar e incorporar principios arquitectónicos
- Observación del dominio de los problemas
- Implementar los patrones y estrategias vistas



Conceptos clave a recordar

- Características arquitectónicas
- Estilos y patrones de arquitectura
- Performance, escalabilidad, integrabilidad, seguridad
- Integración, Data, Testing y monitoreo
- Uso de cloud y métodos de visualización

Planificación estratégica



**It doesn't matter how many resources you have
if you don't know how to use them, they will never be enough**



Principios a recordar

- Todo es un tradeoff
- Las arquitecturas son combinables
- Measure twice, cut once
- Prepárense para el cambio
- Monitorear exhaustivamente



Principios de mantenimiento

- Mejorar costos de infraestructura
- Mejorar Arquitectura / Infraestructura
- Mejorar Observabilidad
- Implementar pipeline CI/CD más robustos
- Migración de bases de datos
- Mejorar la seguridad
- Documentación
- Ingresar nuevos requerimientos



Que hay más allá!

- DDD Deep, ATAM
- Devops y MLops
- 12 FactorApp
- Certificados en cloud computing
- Incident management
- DevSecOps
- Site Reliability Engineering
- Evolutionary architectures
- Zero Trust environments

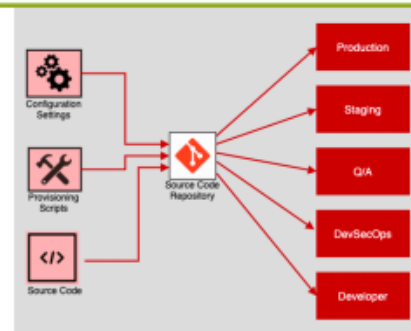
Conjuntos interesantes: 12 factor App

I. Código base (Codebase)

Un código base sobre el que hacer el control de versiones y múltiples despliegues

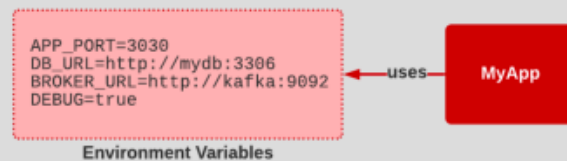
II. Dependencias

Declarar y aislar explícitamente las dependencias



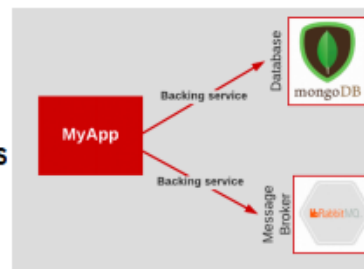
III. Configuraciones

Guardar la configuración en el entorno



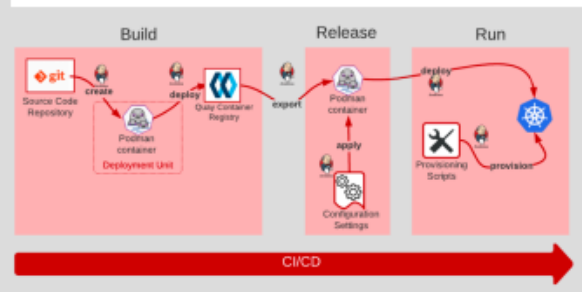
IV. Backing services

Tratar a los “backing services” como recursos conectables



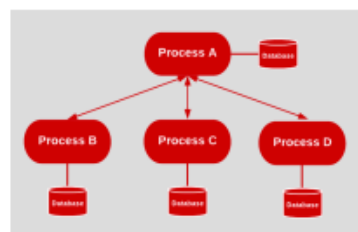
V. Construir, desplegar, ejecutar

Separar completamente la etapa de construcción de la etapa de ejecución



VI. Procesos

Ejecutar la aplicación como uno o más procesos sin estado



12 Factor app

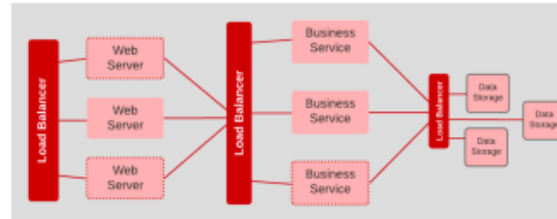
VII. Asignación de puertos

Publicar servicios mediante asignación de puertos



VIII. Concurrencia

Escalar mediante el modelo de procesos

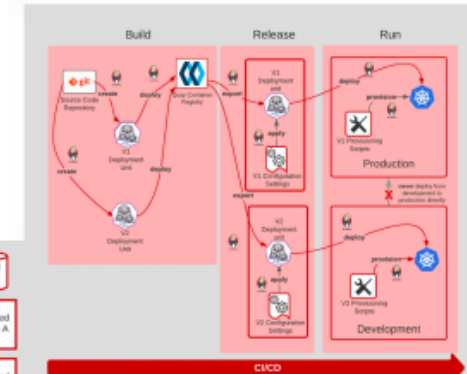


IX. Desechabilidad

Hacer el sistema más robusto intentando conseguir inicios rápidos y finalizaciones seguras

X. Paridad en desarrollo y producción

Mantener desarrollo, preproducción y producción tan parecidos como sea posible



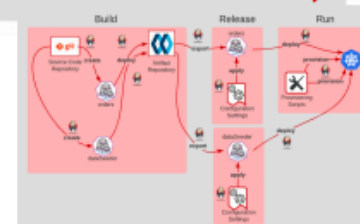
XI. Historiales

Tratar los historiales como una transmisión de eventos



XII. Administración de procesos

Ejecutar las tareas de gestión/administración como procesos que solo se ejecutan una vez



AWS Well Architected Framework



Operational Excellence
Running & monitoring
systems



Security
Protecting information &
managing system integrity



Reliability
Performing as intended
correctly & consistently



Performance Efficiency
Using IT & computing
resources efficiently



Cost Optimization
Avoiding unnecessary costs
by controlling resources

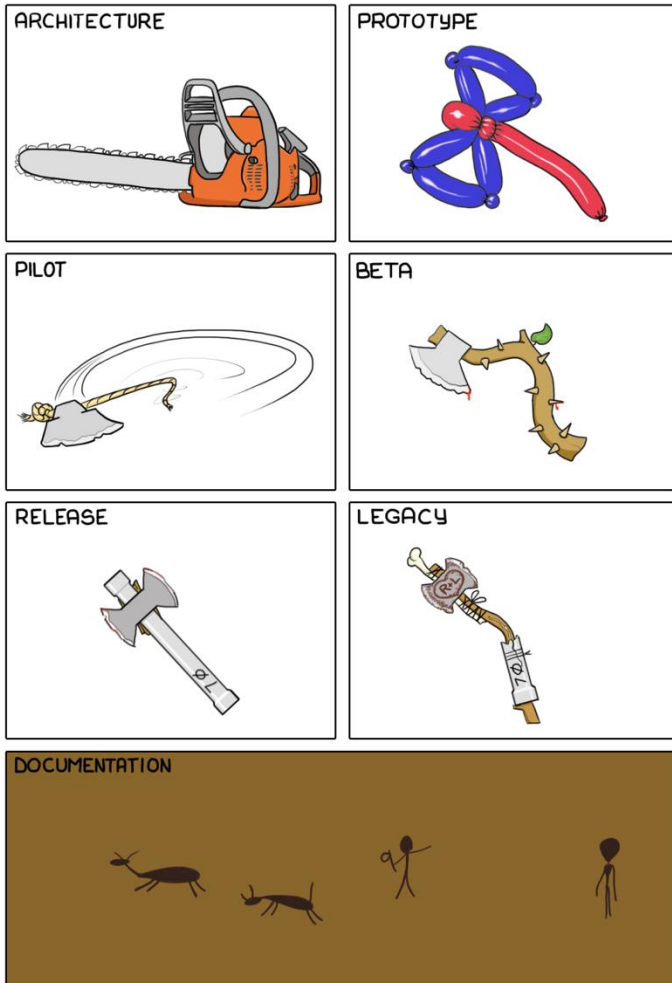


Sustainability
Minimizing environmental
impacts of cloud workloads

<https://aws.amazon.com/architecture/well-architected/>

Preocuparse de aspectos estratégicos

CODE PROGRESSION



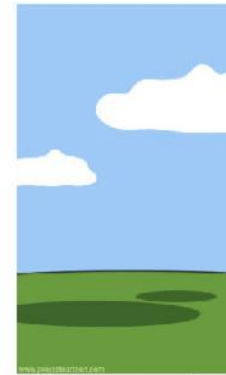
MONKEYUSER.COM



What the customer really needed



How the customer explained it



How the project was documented



How the programmer wrote it



When it was delivered



How the customer was billed

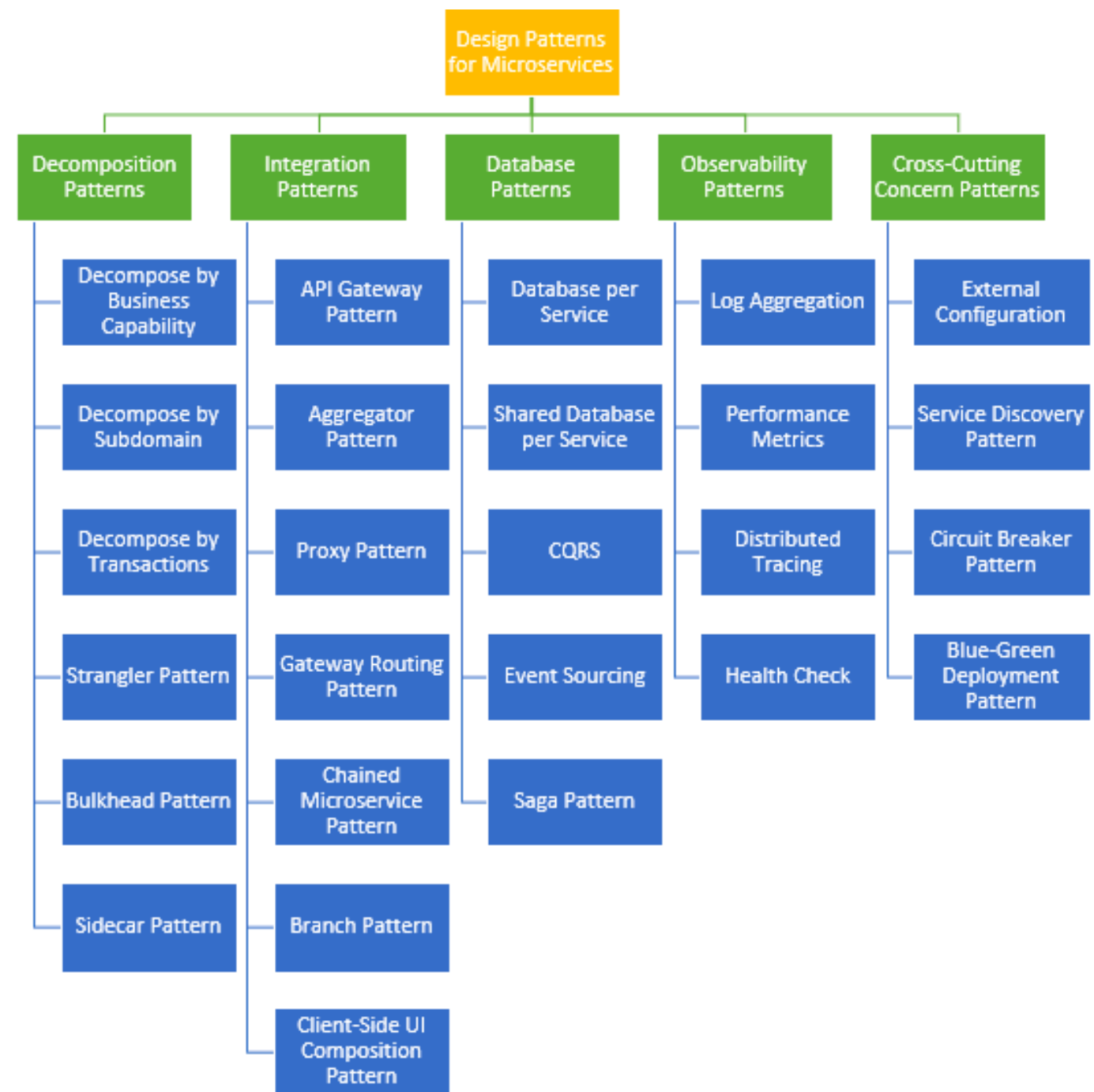


What the digg effect can do to your site



The disaster recover plan

¿Qué más
podemos ver?



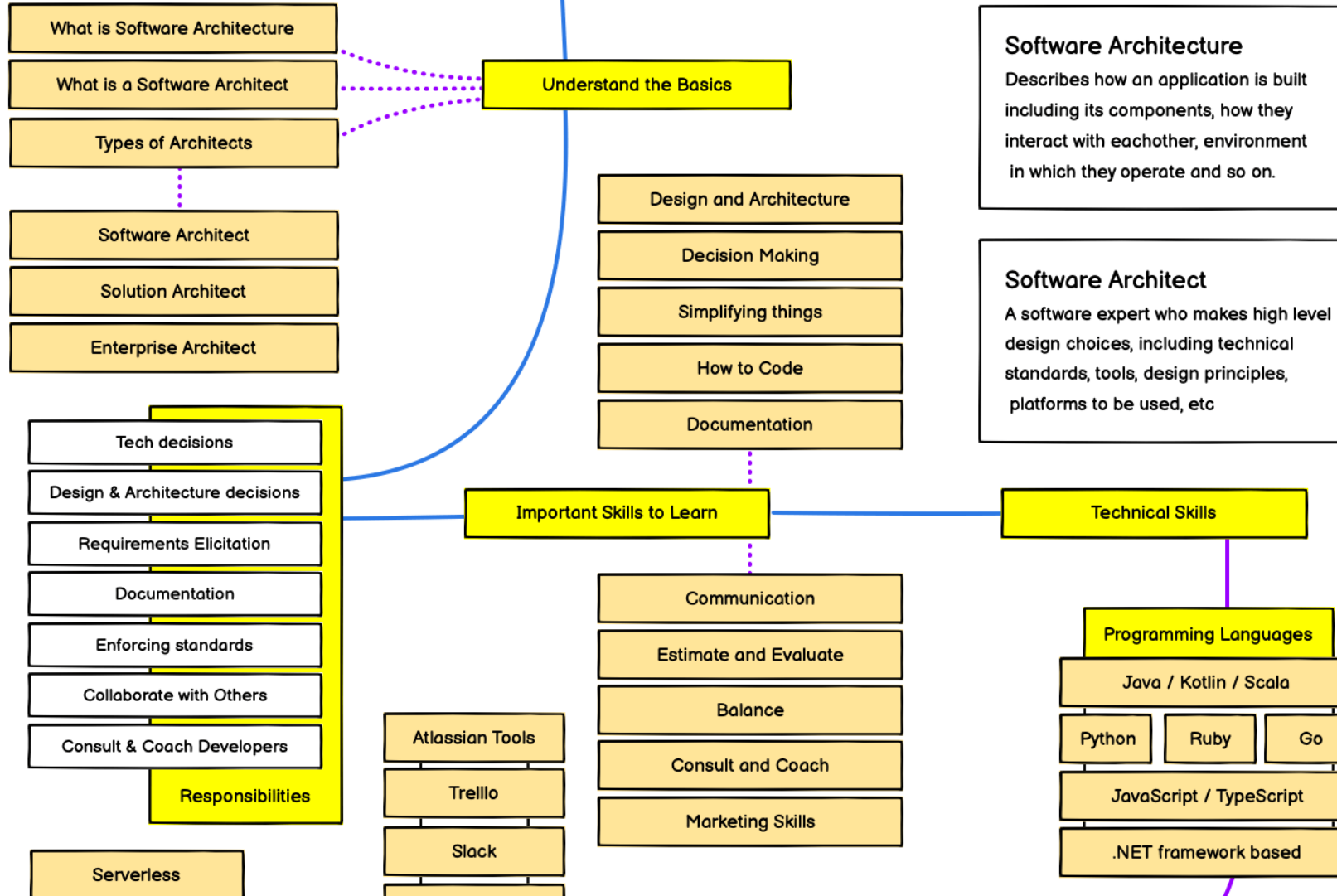
References

- ✓ Justin Miller's Software Architect Guide
- ✓ O'Reilly - Software Architecture

Find the detailed version of this roadmap along with resources, progress tracking and other resources.

<https://roadmap.sh>

Software Architect

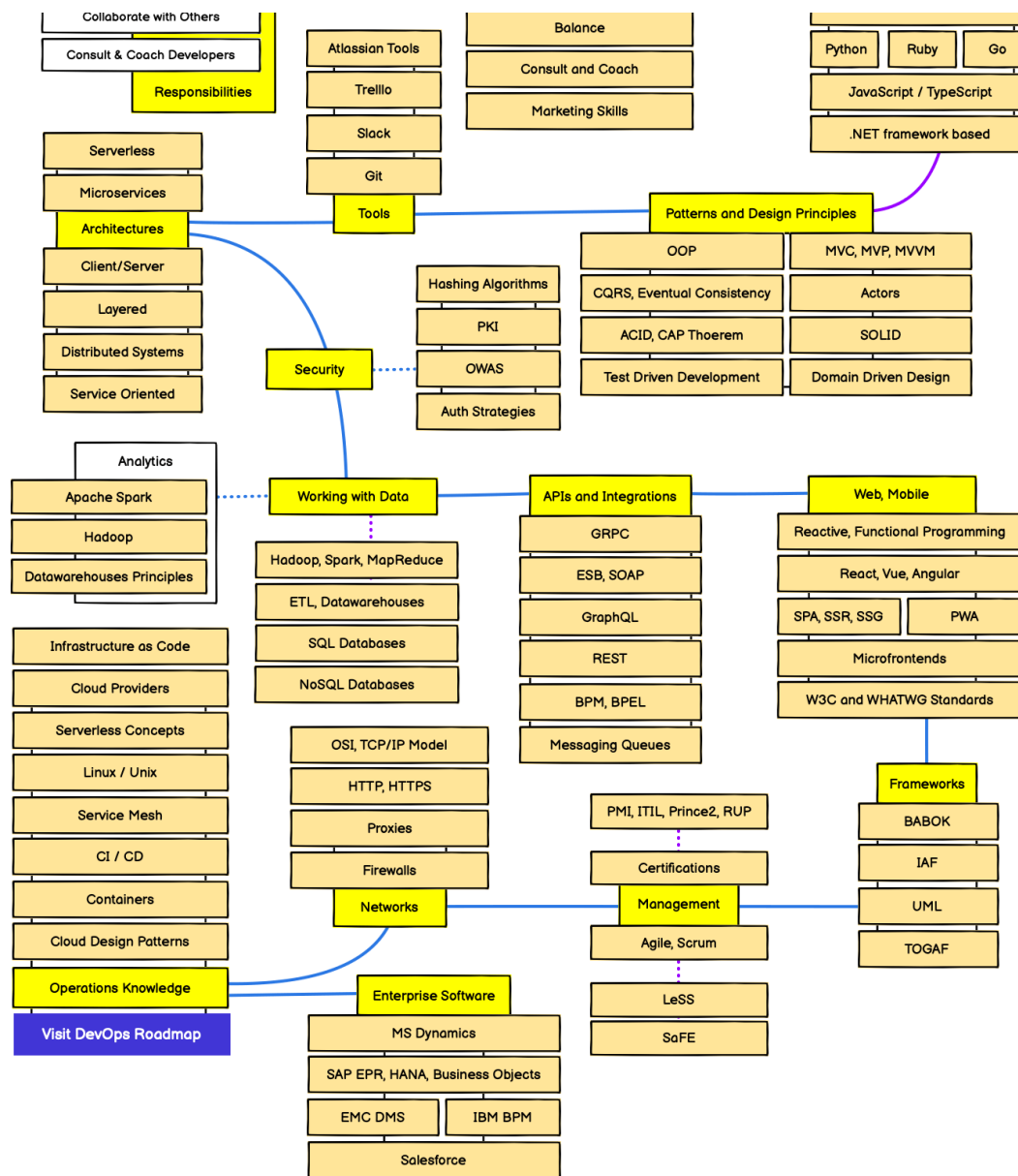


Software Architecture

Describes how an application is built including its components, how they interact with each other, environment in which they operate and so on.

Software Architect

A software expert who makes high level design choices, including technical standards, tools, design principles, platforms to be used, etc



Keep learning



La importancia de la continuidad: Updating

- Revisar teoría esencial: libros y documentos
- Búsqueda de información en papers
- Fuentes establecidas
 - Azure architecture center
- Blogs
 - Martin Fowler
 - InfoQ
 - AWS
 - Medium
 - Microservices.io
- Cursos
 - Universitarios
 - Online

—

¡Gracias por
su atención
en el
semestre!

